Verifiable PIR with Small Client Storage
(Authors' version)

Mayank Rathee
UC Berkeley

Keewoo Lee
UC Berkeley

Raluca Ada Popa
UC Berkeley

***Abstract*—Efficient Verifiable Private Information Retrieval (vPIR) protocols, and more generally Verifiable Linearly Homomorphic Encryption (vLHE), suffer from high client storage. VeriSimplePIR (USENIX Security 2024), the state-of-the-art vPIR protocol, requires clients to persistently maintain over 1 GiB of local storage to privately access an 8 GiB remote database. We present a new vPIR protocol that reduces the client state by orders of magnitude while preserving online latency. In our protocol, clients only need to store 512 KiB for an 8 GiB database, achieving a $2000\times$ improvement. Our vPIR protocol is built over our new vLHE scheme. Unlike VeriSimplePIR, our scheme doesn't use random oracles and relies only on standard lattice assumptions - (R)LWE and SIS. These improvements come at a $2.5\times$ cost in server throughput over VeriSimplePIR. Despite this throughput overhead, we achieve a comparable online latency to VeriSimplePIR by implementing several optimizations including query-level preprocessing. We also introduce the notion of covert vPIR (cvPIR), where stateful clients enjoy full vPIR security, while even stateless clients benefit from covert security against a malicious server.**

1. Introduction

In the classical setting of Private Information Retrieval (PIR) [16], [47], a client seeks to retrieve an entry from a remote database held by an untrusted server while preserving the privacy of their query. Despite significant advances in PIR, most efforts have focused on the semi-honest setting. However, many applications demand PIR schemes that remain secure even against a *malicious* server. For instance, in a public-key directory, a compromised server could respond to a user's request with its own public key to mount a man-in-the-middle attack. Alternatively, the server might try to deduce the user's query by deliberately altering keys in the directory at *suspected* indices. By observing the user's reaction (e.g., aborting if the response isn't a well-formatted key), the server could exploit this behavior in what is known as the *selective-failure* attack [17], [45], [48]. Other vulnerable PIR applications include leaked-password detection [24], auditing in certificate transparency [41], and privately accessing DNS entries [17].

VeriSimplePIR [24] is a state-of-the-art *verifiable* PIR scheme that is secure against a malicious server. It defends against selective-failure attacks and ensures that all responses are consistent with respect to some fixed *extractable* database. However, it suffers from a significant drawback - high client storage. This issue directly contradicts one of the primary motivations for using PIR instead of locally storing the database - limited client storage. For instance, VeriSimplePIR requires over 1 GiB of persistent client storage to access an 8 GiB database and over 400 MiB for a 1 GiB database. This issue becomes even more pronounced when considering a mobile device running k applications, each using PIR to query its own remote database. If each database is 8 GiB or larger, then the client must allocate at least k GiB of storage exclusively for PIR. This imposes a significant constraint on the number of such applications the device can support.

1.1. Our Contribution

(1) vPIR with small client storage. We present a new Verifiable PIR (vPIR) scheme that effectively addresses the issue of high client storage while relying solely on standard lattice assumptions – (R)LWE and SIS. Notably, this is achieved while maintaining the online latency of VeriSimplePIR [24].

- **Small online state:** Conceptually, we eliminate the dependence of the LWE lattice parameter n (typically ≥ 1024) from the size of the client's persistent state. In concrete terms, this reduces client storage by orders of magnitude over VeriSimplePIR; for an 8 GiB database, our scheme requires the client to retain only 512 KiB of state, whereas VeriSimplePIR requires over a GiB. We note that some sort of database *commitment* must be stored on the client side for any vPIR scheme, meaning vPIR without any persistent client storage is impossible. Besides the persistent state, the online short-term state of the client—the total state when actively making queries[1]—remains below 11 MiB in our scheme, compared to over a GiB in VeriSimplePIR for an 8 GiB database. A client who allocates 4 GiB of local storage exclusively for PIR can support about 3 applications using VeriSimplePIR, whereas our protocol can support more than 300. See §1.2 for an overview of our techniques and §6 for detailed experimental results.
- **Small end-to-end state:** The offline phase of our scheme is a streaming-friendly interactive protocol, akin to streaming interactive proofs [18]–[20], and is

1. It is common for software applications to use an extra ephemeral cache on top of persistent state when actively being used.

executed only once. Therefore, it can be streamed while maintaining low state throughout (see §5). Combined with our small online state, this results in a scheme with a small end-to-end state.

Along the way, we demonstrate that neither the Fiat-Shamir heuristic nor a random oracle is necessary for efficient vPIR. We eliminate both while maintaining the round complexity of VeriSimplePIR (see Remark 3.12).

vLHE with small client storage. Our vPIR scheme arises as a corollary of our new Verifiable Linear Homomorphic Encryption (vLHE) scheme, from which it directly inherits all the desirable properties. Beyond vPIR, vLHE has broader applications, including Private Nearest Neighbor Search (PNNS) [8], [22], [40], [52], [63]. Research [38], [67] indicates that large-scale misinformation propagation is possible if the recommendation results are manipulated. This risk can be mitigated by adopting Verifiable PNNS.

(2) Covert security for transient stateless clients. As mentioned earlier, achieving malicious security against a cheating server requires clients to be stateful. In many real-world settings, client-side statefulness is either impractical or undesirable—even when the required state is small, as in our work. This is because in both our scheme and VeriSimplePIR, the state setup may take hundreds of seconds (see §A.5). While frequent users (long-term clients) can amortize this one-time cost over many queries they make over their lifetime, this is a significant burden for *infrequent* users. Motivated by this, we ask:

Is it possible to provide security assurances that go beyond the semi-honest model to stateless clients?

In this work, we answer this question affirmatively in a multi-client model, where some clients are stateful and others are stateless. This model reflects what we see in practice—databases typically have a mix of frequent and infrequent users[2]. Looking ahead, the server should not be able to distinguish whether a query comes from a stateful or stateless client. In this model, we introduce the notion of Covert vPIR (cvPIR), which, broadly speaking, satisfies the following properties:

- Stateful clients enjoy the strong vPIR security.
- The query of a stateless client looks indisinguishable from a stateful client. Therefore, stateless clients can hide among stateful clients, and enjoy *covert* security [9] against a malicious server.
- If the server cheats and is caught by a stateful client, the cheating can be publicly verified.

We present a cvPIR (cvLHE) scheme that extends our vPIR scheme. As a bonus, since the server responds identically to both stateful and stateless queries (ensured by their indistinguishability), our scheme allows resource-constrained clients to opt out of full malicious security without requiring the server to operate two separate systems. See §1.2 for an overview of our cvPIR and §4 for detailed discussions.

2. Even for databases without frequent users, each client can flip a coin which decides if they will be stateful or stateless.

(3) Implementation and optimizations. We implement our protocol in C++ and evaluate its performance against VeriSimplePIR [24], the state-of-the-art in verifiable PIR. Along the way, we also introduce optimizations that were not explored in prior works [24], [41], [51]. These optimizations enhance the efficiency of our protocol across key metrics such as communication, computation, and online latency (see Sections 5 and 6). A prime example is *query-level preprocessing*, which generates a consumable (non-reusable) state for the online phase, similar to how correlated randomness (e.g., Beaver triples) is consumed in the online phase of MPC protocols. By leveraging query-level preprocessing, we significantly enhance online latency. The additional state required to store the output of query-level preprocessing is under 1 MiB per query for an 8 GiB database. With query-level preprocessing, our scheme achieves $8.5\times$ improved online latency for 8 GiB database, while still achieving more than $700\times$ reduction in storage overhead compared to VeriSimplePIR. For a fair comparison, we extend query-level preprocessing to prior work as well, and observe that our scheme achieves comparable online latency and $2.5\times$ overhead in server throughput over VeriSimplePIR.

Morever, we observe that verifiability comes at a small cost in both online latency and server throughput over HintlessPIR [51], a *non*-verifiable PIR scheme over which we build our vPIR scheme. For an 8 GiB database, our latency overhead is $1.6\times$ and server throughput is comparable.

1.2. Technical Overview

Background: SimplePIR and VeriSimplePIR. SimplePIR [41] is based on a variant of Regev's LWE cryptosystem [60] that is tailored to the preprocessing model. Each ciphertext is a tuple $(\boldsymbol{A}, \boldsymbol{u})$ where $\boldsymbol{A}$ is a random matrix and $\boldsymbol{u}$ is a pseudorandom vector. To homomorphically apply a linear transformation $\boldsymbol{D}$ on a ciphertext $(\boldsymbol{A}, \boldsymbol{u})$, one simply computes $(\boldsymbol{D} \cdot \boldsymbol{A}, \boldsymbol{D} \cdot \boldsymbol{u})$. As observed in SimplePIR, a vast majority of the cost of this homomorphic operation comes from computing $\boldsymbol{D} \cdot \boldsymbol{A}$, which can be preprocessed by fixing the matrix $\boldsymbol{A}$ a priori. However, the *hint* $\boldsymbol{H} = \boldsymbol{D} \cdot \boldsymbol{A}$ needs to be stored for use in decryption later and this increases the storage requirements substantially.

VeriSimplePIR [24] adds verifiability to SimplePIR by repurposing the hint $\boldsymbol{H}$ as a short integer solution (SIS) commitment to the database $\boldsymbol{D}$. The protocol proceeds in two phases. In the offline phase, the client sends an encrypted challenge $\boldsymbol{C}$ to the server and receives a proof $\boldsymbol{Z} = \boldsymbol{C} \cdot \boldsymbol{D}$ that is used to verify the server's responses in the online phase. The client first validates the proof against the hint with several checks including $\boldsymbol{Z} \cdot \boldsymbol{A} = \boldsymbol{C} \cdot \boldsymbol{H}$. Then in the online phase, given the client's (encrypted) query $\boldsymbol{u}$, it verifies the server's (encrypted) response $\boldsymbol{v}$ with the check: $\boldsymbol{Z} \cdot \boldsymbol{u} = \boldsymbol{C} \cdot \boldsymbol{v}$. If the verification succeeds, the client uses $\boldsymbol{H}$ to decrypt the response $\boldsymbol{v}$ as $\boldsymbol{v} - \boldsymbol{H} \cdot \boldsymbol{s}$, where $\boldsymbol{s}$ is the secret key used for encryption. Note that this is the only point in the online phase where $\boldsymbol{H}$ is used. Thus, if the client can obtain the *correct* $\boldsymbol{h_s} \leftarrow \boldsymbol{H} \cdot \boldsymbol{s}$ without locally computing it, the high storage requirement for $\boldsymbol{H}$ can be eliminated.

Naive approaches. Recently, fully homomorphic encryption (FHE) has been used to securely delegate the computation of $\boldsymbol{h_s}$ to the server [40], [51], [56] (see §2.5). However, these protocols assume honest-but-curious servers and do not offer verifiability. A straightforward approach to add verifiability would be to use a general SNARK to prove the integrity of this homomorphic computation with respect to a *succinct* commitment to $\boldsymbol{H}$, or to employ more specialized techniques from *verifiable FHE* [7], [12], [32], [44], [46], [64]. However, these approaches introduce a significant overhead.

Another natural approach for verifiably delegating $\boldsymbol{h_s}$ to the server is to recursively apply VeriSimplePIR with $\boldsymbol{H}$ as the database and $\boldsymbol{s}$ as the client's query. However, this approach fails because the hint for the recursive instance, $\boldsymbol{H_H}$, is not smaller than $\boldsymbol{H}$, and the client must store it. With VeriSimplePIR, the number of columns in the hint cannot be compressed below the lattice parameter n (typically 1024-2048).

Our main ideas for vLHE. We now present the key ideas behind our vLHE scheme that achieves small persistent state. See §3 for details.

Black-box verification of FHE evaluation. Given the substantial overhead associated with *white-box* verification of FHE operations, we explore the potential for a more efficient *black-box* verification approach, i.e. to verify the decrypted result rather than the exact operations the server performs over the ciphertext. In other words, the white-box approach verifies *over* encryption, as done in VeriSimplePIR, while the black-box approach aims to verify *under* encryption, i.e. *after* decryption. As we discuss next, black-box verification offers great simplicity, however we have to be careful in using it because now we are verifying some property about a message that was encrypted to hide it from the server. If the verification accepts or rejects, one bit of information about this message is leaked to the server (a.k.a selective failure attack). In our case, this leakage will only be about an ephemeral and high entropy secret-key, not about the private query, making it easier to nullify the attack's damage.

Black-box verification shows efficiency promise for two reasons. First, we are not using FHE for arbitrary computation. Instead, we are delegating a linear transformation $\boldsymbol{h_s} \leftarrow \boldsymbol{H} \cdot \boldsymbol{s}$, even though the server performs more complex operations at the ciphertext level to achieve this efficiently. Second, VeriSimplePIR already verifies if the server correctly applied the linear transformation $\boldsymbol{D}$, which is also linearly related to $\boldsymbol{H}$. Building on this insight, we observe that the verification of $\boldsymbol{h_s} = \boldsymbol{H} \cdot \boldsymbol{s}$ can be reduced to verifying if the linear transformation $\boldsymbol{D}$ was correctly applied to an "imaginary" query. More precisely, since $\boldsymbol{H} = \boldsymbol{D} \cdot \boldsymbol{A}$, we have $\boldsymbol{H} \cdot \boldsymbol{s} = \boldsymbol{D} \cdot \boldsymbol{A} \cdot \boldsymbol{s} = \boldsymbol{D} \cdot \boldsymbol{a_s}$, where $\boldsymbol{a_s} = \boldsymbol{A} \cdot \boldsymbol{s}$ is an imaginary query that the client never explicitly makes. When the client decrypts the server's response to reconstruct $\boldsymbol{h_s}$, it treats this as the server's LHE response to the imaginary query $\boldsymbol{a_s}$. Since the same preprocessed proof $\boldsymbol{Z}$ in VeriSimplePIR can verify multiple LHE query-response pairs with respect to the database $\boldsymbol{D}$, we obtain an efficient method to verify our FHE module. That is, the client only has to perform the following augmented check in the online phase: $\boldsymbol{Z} \cdot [\boldsymbol{u} \quad \boldsymbol{a_s}] = \boldsymbol{C} \cdot [\boldsymbol{v} \quad \boldsymbol{h_s}]$. See §2.5 for details.

During delegation, the client sends encryption of $\boldsymbol{s}$, receives encryption of $\boldsymbol{h_s}$, and performs black-box verification $\boldsymbol{Z} \cdot \boldsymbol{a_s} = \boldsymbol{C} \cdot \boldsymbol{h_s}$. Since $\boldsymbol{s}$ is freshly sampled with each query, given the robustness of (R)LWE under leaky secret [35], we can increase the entropy of $\boldsymbol{s}$ to nullify the impact of the selective failure attack. Importantly, this property of query-independent (FHE-based) delegation is crucial for secure black-box verification. Advanced delegation, as used by YPIR [56], doesn't have query-independence, and therefore, our ideas for vLHE don't extend there.

Compressing the proof. Our ideas so far make the client's persistent state independent of the hint $\boldsymbol{H}$. While this already reduces the state dramatically, the proof $\boldsymbol{Z}$ that remains in the persistent state can still be about 16 MiB for an 8 GiB database.[3] While we cannot fully eliminate the proof because some database-dependent state is necessary for malicious security, we present our idea for proof compression that reduces the persistent state by 10-20$\times$ in typical settings. Our observation is that all the checks the client performs on $\boldsymbol{Z}$ are *linear* in $\boldsymbol{Z}$. Therefore, the checks can be compressed using random linear combinations. As a result, the client no longer needs to store the full matrix $\boldsymbol{Z}$ and instead only stores a few random linear combinations of its rows. See §3.4 and §A.1 for details.

Simpler protocol. Along the way, we also introduce several simplifications to VeriSimplePIR.

- *Simulation-based definition*: We provide a straightforward ideal functionality for vLHE and a simulation-based definition. In contrast, VeriSimplePIR [24, Definition A.4] uses a more complex definition for query-hiding against a malicious server. See §3.1.
- *Removal of Fiat-Shamir heuristic*: VeriSimplePIR recursively applies its own vLHE scheme during the preprocessing phase to verify that the server correctly computes the proof $\boldsymbol{Z} = \boldsymbol{C} \cdot \boldsymbol{D}$, and uses Fiat-Shamir [31] to minimize interaction. We propose a simpler preprocessing phase that avoids the recursive application of vLHE and the use of the Fiat-Shamir while preserving the same round complexity. Our observation is that the check $\boldsymbol{Z} \cdot \boldsymbol{A} = \boldsymbol{C} \cdot \boldsymbol{H}$, primarily used to guarantee $\boldsymbol{H} = \boldsymbol{D} \cdot \boldsymbol{A}$, also ensures $\boldsymbol{Z} = \boldsymbol{C} \cdot \boldsymbol{D}$. This becomes evident by substituting $\boldsymbol{H}$ in the check, yielding $\boldsymbol{Z} \cdot \boldsymbol{A} = \boldsymbol{C} \cdot \boldsymbol{H} = (\boldsymbol{C} \cdot \boldsymbol{D}) \cdot \boldsymbol{A}$. The trade-off is that we require a slightly larger parameter for SIS hardness, due to a new bound on $\boldsymbol{Z}$-related terms that exceeds the bound for $\boldsymbol{D}$. This has a negligible impact on performance under practical parameters. See §3.3.
- *Reusability of proof after verification failure*: We show that if the proof $\boldsymbol{Z}$ is generated during the preprocessing phase without aborting, it can be safely reused in the online phase, even if online verification fails for some queries. In contrast, VeriSimplePIR requires

3. Note that $\boldsymbol{A}$ can be expanded from its seed in a streaming fashion, so it doesn't affect the persistent state.

rerunning preprocessing phase whenever online proof verification fails to maintain security. See Remark 5.1.

cvPIR: Covert security for stateless clients. Our idea to improve the security of stateless clients is to (1) make them indistinguishable from a stateful client, and (2) create a strong deterrent for the server to never cheat. If the server never cheats, stateless clients are no longer vulnerable to selective failure attacks.

For indistinguishability, we observe that the client's query in the online phase of our vLHE scheme is already independent of the state maintained by a stateful client. Therefore, a stateless client can make online queries that look identical to a stateful client, while hiding its IP address via an anonymizing proxy. The server won't be able to tell if the querying client is stateful or stateless. However, since we are entirely relying on the server behaving honestly for the security of stateless clients, we need to create a strong deterrent against cheating. So far, if the server cheats and the query is from a stateless client, then the server potentially learns some private information, while if the client is stateful, then the server is caught by the client. By additionally requiring that the stateful client can convince anyone if the server cheats, we establish severe consequences of cheating—a single cheating can publicly incriminate the server. Therefore, the server defaults to behaving honestly for *all* queries given the possibility of the querying client being stateful. To achieve publicly-verifiable cheating, the server is asked to publicly provide $\boldsymbol{D}$; we also propose a stronger scheme that works without access to $\boldsymbol{D}$ in §A.3. Given $\boldsymbol{D}$, the server's computation is deterministic and anyone can verify its correctness. We have to be careful with malicious clients who might try to frame an honest server. We deal with this issue using commitments and signatures, and ensure that our use of signatures doesn't conflict with client anonymity. See §4 and §A.3 for details.

1.3. Related Work

PIR. Since its introduction in the foundational work of Chor et al. [16], numerous PIR schemes have been proposed [4], [6], [21], [23], [25], [40]–[42], [48], [51], [54]–[57], [69], many of which are efficient enough for deployment [27], [43], [49]. However, PIR research has focused primarily on ensuring privacy against an honest-but-curious server.

PIR against malicious server. Since the problem was first posed in the seminal work of Kushilevitz and Ostrovsky [48], several attempts have been made over the years to construct PIR schemes that are secure against malicious servers. Some schemes [11], [65], [68] guarantee that the server's response to each query is honestly generated with respect to *some* (extractable in [11]) database. However, they do not ensure consistency across the databases used for each query, leaving them vulnerable to selective failure attacks. Recently, Colombo et al. [17] and Wang et al. [66] constructed PIR schemes that are secure against a malicious server under the assumption of an *honestly* generated commitment to the "original" database. This honest commitment assumption has since been relaxed in subsequent works [24], [26], [29]. However, Dietz and Tessaro [26] and Falk et al. [29] present constructions without implementing their schemes. Currently, VeriSimplePIR [24] and Crust [66] are the most efficient vPIR constructions.

As a follow-up to Piano [69], Crust requires all clients to stream the full database from the server before making their first query, imposing a substantial communication burden on onboarding clients. Additionally, since clients retain state from this process, Crust cannot support stateless clients, even under an honest-but-curious server, thus failing to achieve cvPIR. Anonymous queries are also unsupported, as each client must provide a client-specific pointer to the server to refresh its state.

2. Preliminaries

Notation. All logarithms $\log(\cdot)$ are base 2. We use $[\ell]$ to denote the set $\{1, \ldots, \ell\}$ and $\{x_i\}_{i\in[n]}$ to denote the set $\{x_1, \ldots, x_n\}$. A uniform sample x from a finite set $\mathcal{S}$ is denoted by $x \xleftarrow{\$} \mathcal{S}$, while a sample from the probability distribution χ is denoted by $x \leftarrow \chi$. The distribution induced by a random variable X is denoted by $\{X\}$. For a vector $\boldsymbol{v}$, its i-th entry is denoted by $\boldsymbol{v}[i]$. We use $\| \cdot \|_\infty$ to denote the infinity-norm of a vector or a matrix. We denote the floor function by $\lfloor \cdot \rfloor$. For $x \in \mathbb{R}$, the round function $\lfloor x \rceil$ is defined as $\lfloor x + \frac{1}{2} \rfloor$. For $a \in \mathbb{Z}$, $[a]_q$ is defined as $a - \lfloor \frac{a}{q} \rceil \cdot q$. These notations are naturally extended to vectors, where the operations are applied componentwise. We denote a ciphertext encrypting a message μ as $\mathtt{ct}(\mu)$. This notation is also extended to denote a list of ciphertexts encrypting a corresponding list of messages. We use $\mathsf{poly}(n)$ (resp. $\mathsf{ngl}(n)$) to denote polynomial (resp. negligible) functions in n. We use κ to denote the statistical security parameter and λ to denote the computational security parameter. We denote $\equiv_\lambda$ for computational indistinguishability w.r.t λ.

2.1. Lattice Problems: LWE and SIS

The Learning with Errors (LWE) problem [61] is defined as follows. It has four parameters: dimension n, number of samples m, modulus q, and error distribution χ over $\mathbb{Z}_q$.

Definition 2.1 ((n, m, q, χ)-LWE)**.** *Distinguish between two distributions:*

- $(\boldsymbol{A}, \boldsymbol{A}\boldsymbol{s} + \boldsymbol{e})$*, where* $\boldsymbol{A} \xleftarrow{\$} \mathbb{Z}_q^{m\times n}$*,* $\boldsymbol{s} \xleftarrow{\$} \mathbb{Z}_q^n$*,* $\boldsymbol{e} \leftarrow \chi^m$
- $(\boldsymbol{A}, \boldsymbol{r})$*, where* $\boldsymbol{A} \xleftarrow{\$} \mathbb{Z}_q^{m\times n}$ *and* $\boldsymbol{r} \xleftarrow{\$} \mathbb{Z}_q^m$

The Short Integer Solution (SIS) problem [1] is defined as follows. The problem has four parameters: dimension n and m, modulus q, and bound β.

Definition 2.2 ((n, m, q, β)-SIS)**.** *Given* $\boldsymbol{A} \xleftarrow{\$} \mathbb{Z}_q^{n\times m}$*, find non-zero* $\boldsymbol{x} \in \mathbb{Z}^m$ *of norm* $\|\boldsymbol{x}\|_\infty \leq \beta$ *s.t.* $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{0} \in \mathbb{Z}_q^n$*.*

2.2. Fully Homomorphic Encryption

Fully Homomorphic Encryption (FHE) is a cryptosystem that supports computation on encrypted data [33], [62].

Here, we present a syntax for a (symmetric key) FHE scheme with message space $\mathcal{M}$ and ciphertext space $\mathcal{C}$.

- $\mathtt{pp} \leftarrow \mathtt{Setup}(1^\lambda, \mathcal{M})$: The setup algorithm takes security parameter λ and message space $\mathcal{M}$ and outputs public parameters. All the following algorithms take $\mathtt{pp}$ as an implicit input.
- $(\mathtt{sk}, \mathtt{evk}) \leftarrow \mathtt{KeyGen}(\mathtt{pp})$: The key generation algorithm outputs a secret key $\mathtt{sk}$ (and an evaluation key $\mathtt{evk}$ if required).
- $\mathtt{ct}(m) \leftarrow \mathtt{Enc}_{\mathtt{sk}}(m)$: The encryption algorithm takes secret key $\mathtt{sk}$ and message $m \in \mathcal{M}$. It outputs a ciphertext $\mathtt{ct}(m) \in \mathcal{C}$ encrypting m.
- $\mathtt{ct}(f(\boldsymbol{m})) \leftarrow \mathtt{Eval}_{\mathtt{evk}}(f, \mathtt{ct}(\boldsymbol{m}))$: The evaluation algorithm takes a circuit f and a list of ciphertexts $\mathtt{ct}(\boldsymbol{m}) \in \mathcal{C}^*$ (and evaluation key $\mathtt{evk}$ if required). It outputs a list of ciphertexts $\mathtt{ct}(f(\boldsymbol{m})) \in \mathcal{C}^*$ that encrypts $f(\boldsymbol{m}) \in \mathcal{M}^*$.
- $m \leftarrow \mathtt{Dec}_{\mathtt{sk}}(\mathtt{ct}(m))$: The decryption algorithm takes secret key $\mathtt{sk}$ and ciphertext $\mathtt{ct}(m)$ as input. Then, it outputs a message m.

An FHE scheme satisfies the following security definitions. Let $\mathtt{pp} \leftarrow \mathtt{Setup}(1^\lambda, \mathcal{M})$ and $(\mathtt{sk}, \mathtt{evk}) \leftarrow \mathtt{KeyGen}(\mathtt{pp})$.

- **Correctness:** For any circuit f and input $\boldsymbol{m} \in \mathcal{M}^*$, the following holds with overwhelming probability for $\mathtt{ct}(\boldsymbol{m}) \leftarrow \mathtt{Enc}_{\mathtt{sk}}(\boldsymbol{m})$:
$$\mathtt{Dec}_{\mathtt{sk}}\left(\mathtt{Eval}_{\mathtt{evk}}(f, \mathtt{ct}(\boldsymbol{m}))\right) = f(\boldsymbol{m})$$
- **Semantic Security:** For any probabilistic polynomial time adversary $\mathcal{A}$ and $m_1, m_2 \in \mathcal{M}$, the following holds for $\mathtt{ct}_i \leftarrow \mathtt{Enc}_{\mathtt{sk}}(m_i)$:
$$|\Pr[\mathcal{A}(\mathtt{evk}, \mathtt{ct}_1) = 1] - \Pr[\mathcal{A}(\mathtt{evk}, \mathtt{ct}_2) = 1]| \leq \mathtt{ngl}(\lambda)$$

Linearly Homomorphic Encryption. We often consider homomorphic encryption schemes that support a class of circuits rather than arbitrary circuits. Linearly Homomorphic Encryption (LHE) is a cryptosystem that supports linear transformations on encrypted vectors. More precisely, an LHE scheme satisfies the following for any matrix $\boldsymbol{D}$ and vector $\boldsymbol{m}$ over $\mathcal{M}$ with matching dimensions:[4]
$$\mathtt{Dec}_{\mathtt{sk}}\left(\mathtt{Eval}_{\mathtt{evk}}(\boldsymbol{D}, \mathtt{ct}(\boldsymbol{m}))\right) = \boldsymbol{D}\boldsymbol{m}$$

2.3. Private Information Retrieval from LHE

A Private Information Retrieval (PIR) scheme allows a client to privately query a remote database entry. PIR can be constructed from LHE by reshaping the database into a matrix using a well-established technique [47]. The database with N entries is organized into an $\ell \times m$ matrix $\boldsymbol{D}$, where $\ell \cdot m \geq N$. To query the i-th index, the client computes the corresponding row index $i_r \in [\ell]$ and column index $i_c \in [m]$. The client encrypts the i_c-th unit vector of size m using LHE and sends it to the server. The server applies the linear transformation $\boldsymbol{D}$ to the ciphertext and returns the result. The client recovers the i-th database entry by decrypting the i_r-th element in the server's response.

4. Throughout this paper, we slightly abuse notation so that $\boldsymbol{D}$ also denotes a circuit for computing the linear transformation $\boldsymbol{D}$.

2.4. Regev's LWE Cryptosystem and SimplePIR

At a high level, our verifiable PIR scheme is a descendant of SimplePIR [41]. In this section, we review the SimplePIR scheme and Regev's LWE cryptosystem [60], on which SimplePIR is based.

Regev's LWE Cryptosystem. Here, we present a symmetric key, vectorized version of Regev's scheme $(\mathtt{Setup}, \mathtt{KeyGen}, \mathtt{Enc}, \mathtt{Dec})$. We also define encoding/decoding subroutine $\mathtt{Ecd}$ and $\mathtt{Dcd}$ for later use.

- $\mathtt{pp} \leftarrow \mathtt{Setup}(1^\lambda, p)$: The setup algorithm takes security parameter λ and plaintext modulus p and outputs public parameter $\mathtt{pp}$ which specifies LWE parameters (n, q, χ). All the following algorithms take $\mathtt{pp}$ as an implicit input.
- $\mathtt{sk} \leftarrow \mathtt{KeyGen}(\mathtt{pp})$: The key generation algorithm outputs a secret key $\mathtt{sk} = \boldsymbol{s} \xleftarrow{\$} \mathbb{Z}_q^n$.
- $\mathtt{ct}(\boldsymbol{\mu}) \leftarrow \mathtt{Enc}_{\mathtt{sk}}(\boldsymbol{\mu})$: The encryption algorithm takes secret key $\mathtt{sk} = \boldsymbol{s} \in \mathbb{Z}_q^n$ and message $\boldsymbol{\mu} \in \mathbb{Z}_p^m$ as input. Then, it samples matrix $\boldsymbol{A} \xleftarrow{\$} \mathbb{Z}_q^{m \times n}$ and error $\boldsymbol{e} \leftarrow \chi^m$. Finally, it outputs $\mathtt{ct}(\boldsymbol{\mu}) = (\boldsymbol{A}, \boldsymbol{A}\boldsymbol{s} + \boldsymbol{e} + \mathtt{Ecd}_{p,q}(\boldsymbol{\mu}))$ as a ciphertext of $\boldsymbol{\mu}$, where $\mathtt{Ecd}_{p,q}$ is defined below.
- $\boldsymbol{\mu}' \leftarrow \mathtt{Ecd}_{p,q}(\boldsymbol{\mu})$: The encoding subroutine with parameters p and q takes a $\mathbb{Z}_p$-vector $\boldsymbol{\mu}$ as input and outputs a $\mathbb{Z}_q$-vector $\lfloor \frac{q}{p} \rfloor \cdot [\boldsymbol{\mu}]_p$.
- $\boldsymbol{\mu} \leftarrow \mathtt{Dec}_{\mathtt{sk}}(\mathtt{ct}(\boldsymbol{\mu}))$: The decryption algorithm takes secret key $\mathtt{sk} = \boldsymbol{s} \in \mathbb{Z}_q^n$ and ciphertext $\mathtt{ct}(\boldsymbol{\mu}) = (\boldsymbol{A}, \boldsymbol{b})$ as input. It computes $\boldsymbol{b}' \leftarrow \boldsymbol{b} - \boldsymbol{A}\boldsymbol{s}$ and outputs $\boldsymbol{\mu} = \mathtt{Dcd}_{p,q}(\boldsymbol{b}')$, which is defined below.
- $\boldsymbol{\mu} \leftarrow \mathtt{Dcd}_{p,q}(\boldsymbol{b}')$: The decoding subroutine with parameters p and q takes a $\mathbb{Z}_q$-vector $\boldsymbol{b}'$ as input and outputs a $\mathbb{Z}_p$-vector $\lfloor \frac{p}{q} \cdot [\boldsymbol{b}']_q \rceil$.

The semantic security directly follows from the hardness of the underlying LWE problem, and the correctness holds as long as $\|\boldsymbol{e}\|_\infty < \lfloor \frac{q}{p} \rfloor / 2$.

Remark 2.3 (Linear Homomorphism)**.** Regev's cryptosystem is actually a linearly homomorphic encryption scheme (§2.2). Consider a linear transformation $\boldsymbol{D} \in \mathbb{Z}^{\ell \times m}$ and a ciphertext $\mathtt{ct}(\boldsymbol{\mu}) = (\boldsymbol{A}, \boldsymbol{b})$ encrypting $\boldsymbol{\mu} \in \mathbb{Z}_p^m$. We can compute $\mathtt{ct}(\boldsymbol{D}\boldsymbol{\mu})$ from $\mathtt{ct}(\boldsymbol{\mu})$ by setting $\mathtt{Eval}(\boldsymbol{D}, \mathtt{ct}(\boldsymbol{\mu})) = (\boldsymbol{D}\boldsymbol{A}, \boldsymbol{D}\boldsymbol{b})$. Note that the homomorphically evaluated ciphertext $\mathtt{ct}(\boldsymbol{D}\boldsymbol{\mu})$ decrypts correctly as long as $\|\boldsymbol{D}\boldsymbol{e}\|_\infty < \lfloor q/p \rfloor / 2$ holds for the original error $\boldsymbol{e}$ of $\mathtt{ct}(\boldsymbol{\mu})$. For more details, please refer to Lemma A.2.

SimplePIR. As noted in §2.3, we can construct PIR from LHE by reshaping the database into a matrix. However, directly applying Regev's scheme to construct such a PIR scheme is far from practical. The bottleneck lies in computing $\boldsymbol{D}\boldsymbol{A}$, where $\boldsymbol{A} \in \mathbb{Z}_q^{m \times n}$ also constitutes a significant portion of the query ciphertext $(\boldsymbol{A}, \boldsymbol{b})$. Notably, n is typically set to be greater than a thousand to ensure LWE hardness, making the computation and communication expensive.

The simple yet powerful idea of SimplePIR [41] is to *preprocess* this bottleneck of computing $\boldsymbol{D}\boldsymbol{A}$. Since $\boldsymbol{D}\boldsymbol{A}$

is entirely independent of the query, it can be precomputed in an offline phase. To enable this, SimplePIR samples the matrix $\boldsymbol{A}$ during setup, which is shared across all queries. The server precomputes $\boldsymbol{H} = \boldsymbol{D}\boldsymbol{A}$ as a *hint* and sends it to clients in the offline phase; clients store it for use in decryption. Meanwhile, for security, a fresh secret $\boldsymbol{s}$ must be sampled for each query. This approach greatly enhances server computation and communication, allowing server throughput to reach the memory input/output limit [41].

2.5. "Hintless" PIR

A limitation of SimplePIR (§2.4) is that the decryption process relies on a significantly large *hint* $\boldsymbol{H}$, which the client must store. To address this issue, follow-up works TiptoePIR [40] and HintlessPIR [51] proposed *securely* delegating the computation involving $\boldsymbol{H}$ to the server using FHE, thereby eliminating the need for the client to maintain the hint. We present the framework of these "Hintless" PIR schemes in Figure 1, abstracting various technical details within the syntax of FHE (§2.2). Notably, unlike SimplePIR, the offline phase is executed entirely on the server-side without requiring any communication with the client.

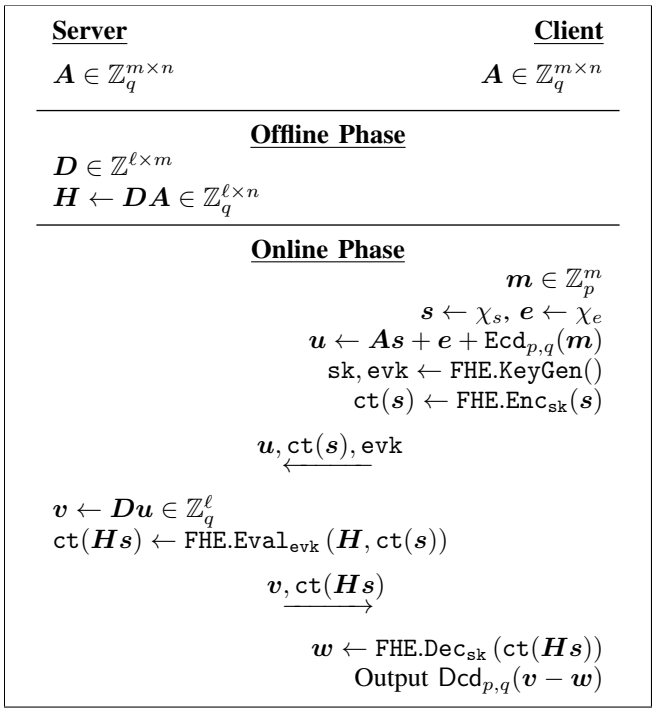

Figure 1: LHE scheme underlying "Hintless" PIR.

2.6. SIS Hash and Proof of Knowledge

SIS Hash. Ajtai's SIS hash [2] allows one to commit to a *short* matrix $\boldsymbol{D} \in \mathbb{Z}^{\ell\times m}$ as $\boldsymbol{H} = \boldsymbol{D}\boldsymbol{A} \in \mathbb{Z}_q^{\ell\times n}$, where $\boldsymbol{A}$ is uniform randomly sampled from $\mathbb{Z}_q^{m\times n}$. This SIS hash is a *binding commitment* for short matrices based on SIS as follows. If two distinct matrices $\boldsymbol{D}_1, \boldsymbol{D}_2$ satisfy $\|\boldsymbol{D}_i\|_\infty \le B$ and $\boldsymbol{H} = \boldsymbol{D}_i\boldsymbol{A}$, then their difference $(\boldsymbol{D}_1 - \boldsymbol{D}_2)$ forms a matrix such that $0 < \|\boldsymbol{D}_1 - \boldsymbol{D}_2\|_\infty \le 2B$ and $(\boldsymbol{D}_1 - \boldsymbol{D}_2)\boldsymbol{A} = \boldsymbol{0}$. This ultimately yields a solution to $(n, m, q, 2B)$-SIS on $\boldsymbol{A}^T$.

Proof of Knowledge for SIS Hash Preimage. Both VeriSimplePIR [24] and our scheme heavily rely on the efficient proof of knowledge for SIS hash preimage proposed by [10]. We describe the protocol in Figure 2.The following lemma states that any prover capable of passing the checks in Figure 2 must *know* some $\boldsymbol{D}'$ satisfying $\|\boldsymbol{D}'\|_\infty \le 2B$.

Lemma 2.4 (Knowledge Soundness [10], [24])**.** *Given a prover $\mathcal{P}$ that passes the checks in Figure 2 with probability $\epsilon > 2^{-\kappa+2}$, we can construct a knowledge extractor $\mathcal{E}$ that invokes the prover $O(\mathsf{poly}(\ell)/\epsilon)$ times to extract $\boldsymbol{D} \in \mathbb{Z}^{\ell\times m}$ satisfying $\boldsymbol{D}\boldsymbol{A} = \boldsymbol{H}$ and $\|\boldsymbol{D}\|_\infty \le 2B$, with overwhelming probability.*

Remark 2.5 (Interactive Proof with Encrypted Challenge)**.** In our protocol (Figure 5), we use a variant of Figure 2, where the challenge $\boldsymbol{C}$ is encrypted under the verifier's key and the prover responds with a homomorphically computed proof. Although parts of the protocol are executed under FHE, the statement of Lemma 2.4 remains exactly the same.

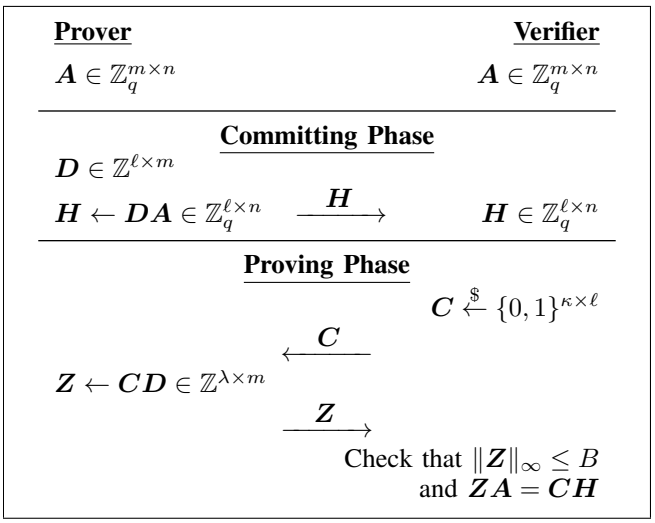

Figure 2: SIS Hash (Commitment) and Proof of Knowledge for Its Preimage [10]. For a purported infinity-norm bound β of $\boldsymbol{D}$, we set $B \ge \ell \cdot \beta$.

3. Verifiable LHE with Small Client Storage

In this section, we present our new Verifiable LHE (vLHE) scheme which significantly reduces the client storage compared to state-of-the-art vLHE scheme from VeriSimplePIR [24]. We begin by introducing a simulation-based definition of vLHE (§3.1). To convey our main ideas in a structured manner, we present the construction step by step across several subsections (Sections 3.2 to 3.4).

3.1. Simulation-based Definition

Definition 3.1 (Offline-online and client-server protocols)**.** *A protocol π is a client-server protocol if there is a set $\{C_i\}_{i\in\mathbb{N}}$ of client parties who all communicate to only a single party, called the server party denoted by S. Moreover, the protocol is called an offline-online protocol if it includes two sub-protocols π_0, π_1, where π_0, the offline protocol, is independent of the clients' input, while π_1, the online protocol, depends on the client's input and the output of π_0.*

Definition 3.2 (vLHE input transcript family)**.** *Given message space $\mathcal{M}$ and $\lambda \in \mathbb{N}$, define $I_{\mathcal{M},k}$ be the family of ordered sets with k instructions per set, where each instruction is one of the following two types:*

1) $\mathsf{Register}$*: Register instruction is of the form* $(\mathsf{id}, \mathsf{Register})$*, denoting the id of the client who registers for the first time.*
2) Query*: Query instruction is of the form* $(\mathsf{id}, \mathsf{Query}, \boldsymbol{x})$*, denoting the client's id who queries with input $\boldsymbol{x} \in \mathcal{M}$.*

Moreover, $(\mathsf{id}, \mathsf{Register})$ *should always appear before* $(\mathsf{id}, \mathsf{Query}, \boldsymbol{x})$ *in the set. Then the input transcipt family $\mathcal{I}_{\mathcal{M},\lambda} = \cup_{k\in\mathbb{Z}_{\mathsf{poly}(\lambda)}} I_{\mathcal{M},k}$.*

Definition 3.3 (Real and ideal executions)**.** *Given a functionality $\mathcal{F}$, a protocol π, non-uniform polynomial-time machines $\mathcal{A}$ and $\mathcal{S}$, input X, auxiliary input z and security parameter λ:*

- *The ideal execution of $\mathcal{F}$ with adversary $\mathcal{S}$, denoted by* $\text{IDEAL}_{\mathcal{F},\mathcal{S}(z)}(X,\lambda)$*, is defined as the set of outputs returned by $\mathcal{F}$ along with the output of $\mathcal{S}$.*
- *The real execution of π with the adversary $\mathcal{A}$ and an implicit set of honest clients $\mathcal{C}$, denoted by* $\text{REAL}_{\pi,\mathcal{A}(z)}(X,\lambda)$*, is defined as the set of outputs returned by the protocol π (to honest parties) along with the output of $\mathcal{A}$.*

Definition 3.4 (vLHE)**.** *Let $\mathcal{F}$ be the ideal functionality defined in Figure 3. An offline-online client-server protocol π (Definition 3.1) with optimal online round complexity*[5] *is a Verifiable Linearly Homomorphic Encryption (vLHE) scheme over the message space $\mathcal{M}$ that securely computes $\mathcal{F}$ with abort in the presence of a malicious server if $\forall$ non-uniform probabilistic polynomial-time (nu-PPT) adversary $\mathcal{A}$, there exists a non-uniform probabilistic polynomial-time adversary $\mathcal{S}$ such that the following holds:*

$$\left\{\text{IDEAL}_{\mathcal{F},\mathcal{S}(z)}(X,\lambda)\right\}_{\substack{X\in\mathcal{I}_{\mathcal{M},\lambda},\\ z\in\{0,1\}^*,\lambda\in\mathbb{N}}} \equiv_\lambda \left\{\text{REAL}_{\pi,\mathcal{A}(z)}(X,\lambda)\right\}_{\substack{X\in\mathcal{I}_{\mathcal{M},\lambda},\\ z\in\{0,1\}^*,\lambda\in\mathbb{N}}}$$

where IDEAL *and* REAL *are defined in Definition 3.3, and $\mathcal{I}_{\mathcal{M},\lambda}$ is the LHE input transcript family as defined in Definition 3.2.*

Remark 3.5 (vPIR from vLHE)**.** In §2.3, we demonstrated how to construct a PIR scheme from LHE. Likewise, a vPIR scheme can be constructed from vLHE using the same approach. Since a vPIR scheme constructed this way is a special case of vLHE, its security properties naturally inherit those of vLHE.

5. vLHE only includes protocols with 2 online rounds.

Ideal Functionality $\mathcal{F}$

Ideal functionality $\mathcal{F}$ with message space $\mathcal{M}$ is a stateful functionality defined as follows:

- On register request $(\mathsf{id}, \mathsf{Register})$:
  - Send $(\mathsf{id}, \mathsf{Register})$ to the adversary.
  - Receive database $\boldsymbol{D} \in \mathcal{M}^\ell$ or Abort from the adversary.
  - If the adversary says Abort, return $\perp$ to the client.
  - Store $\boldsymbol{D}_{\mathsf{id}} \leftarrow \boldsymbol{D}$.
- On query request $(\mathsf{id}, \mathsf{Query}, \boldsymbol{x})$, where $\boldsymbol{x} \in \mathcal{M}$:
  - Send $(\mathsf{id}, \mathsf{Query}, \perp)$ to the adversary. ▷ Query-hiding. For anonymity, see Remark A.4.
  - Receive Allow or Abort from the adversary.
  - If the adversary says Allow, return $\boldsymbol{D}_{\mathsf{id}} \cdot \boldsymbol{x}$ to the client. Otherwise, return $\perp$ to the client.

Figure 3: Ideal functionality for vLHE.

Remark 3.6. Note that our definition considers a single-hop [34] secret-key version of LHE. Although a more general form of vLHE can be defined, this version suffices for our purpose of constructing vPIR.

3.2. vLHE from "Hintless" PIR

A notable limitation of SimplePIR [41] (and VeriSimplePIR [24]) is that the decryption process relies on a significantly large hint $\boldsymbol{H}$, which the client must store. To address this issue, follow-up works TiptoePIR [40] and HintlessPIR [51] proposed *securely* delegating the computation involving $\boldsymbol{H}$ to the server using FHE, thereby eliminating the need for the client to maintain the hint (§2.5). However, extending this approach to the setting of vLHE or vPIR introduces the additional requirement of verifying the server's computations to ensure protocol integrity and prevent malicious behavior.

In this section, we describe how to adapt the idea from TiptoePIR and HintlessPIR to the setting of verifiable LHE. We outline our protocol in Figure 4 and provide its main idea in Lemma 3.7. Our key observation is that the client can verify the server's computation of $\boldsymbol{Hs}$ without relying on proofs the involve the internal structure of the FHE scheme (Remark 3.8). In fact, no additional proof for $\boldsymbol{Hs}$ is needed, as the existing proof $\boldsymbol{Z}$ suffices.

We note that while this is a critical transition enabling subsequent improvements, the resulting scheme is only an intermediate step toward the final solution. Although it removes the need for $\boldsymbol{H}$ in the decryption process, $\boldsymbol{H}$ is still required to verify the server's response, which may appear

meaningless. However, we will later integrate additional techniques (Sections 3.3 and 3.4) to reduce the client's online state significantly. For later usage, we minimize the use of $\boldsymbol{H}$ in the verification procedure, leveraging the fact that $\boldsymbol{Hs} = \boldsymbol{D}(\boldsymbol{As})$. (See Remark 3.9.)

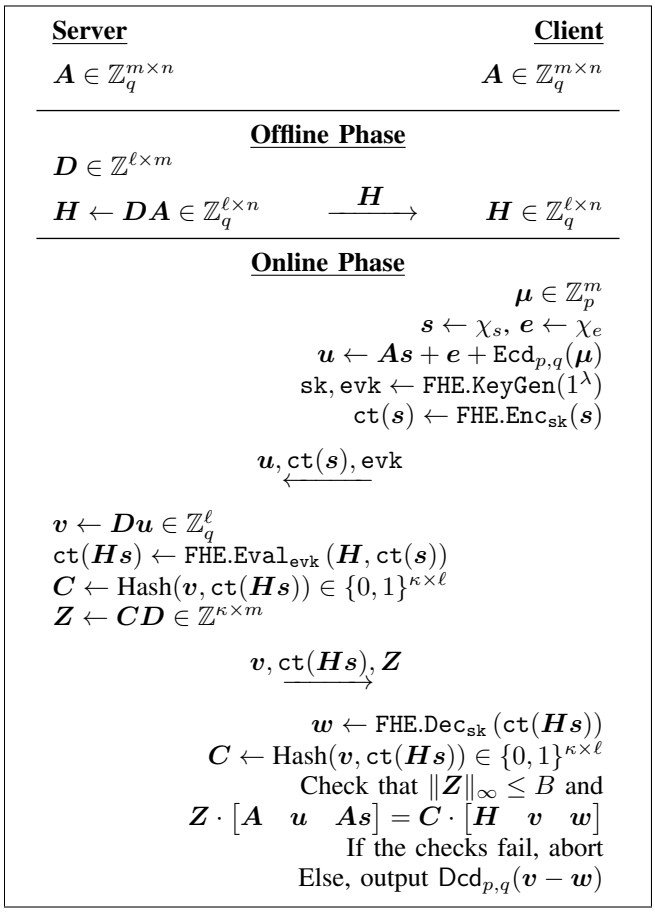

Figure 4: vLHE from Hintless PIR. B is described in Figure 2.

Lemma 3.7. *(Figure 4 is vLHE). Under the random oracle model and $(n, m, q, 4B)$-SIS hardness, the protocol described in Figure 4 is a vLHE scheme (Definition 3.4), as long as q is set so that the underlying Regev's LHE (§2.4) supports linear transformations $\boldsymbol{D}' \in \mathbb{Z}^{\ell \times m}$ with $\|\boldsymbol{D}'\|_\infty \le 2B$ with overwhelming probability*[6].

Proof. (Sketch) As this protocol is only an intermediate step toward the final protocol, we only give a sketch of proof here. The protocol is essentially a hintless PIR (Figure 1) augmented with a proof of knowledge from Figure 2, transformed into a non-interactive form using the Fiat-Shamir heuristic. Applying Lemma 2.4 to the check $\boldsymbol{Z} \cdot \begin{bmatrix} \boldsymbol{A} & \boldsymbol{u} & \boldsymbol{As} \end{bmatrix} = \boldsymbol{C} \cdot \begin{bmatrix} \boldsymbol{H} & \boldsymbol{v} & \boldsymbol{w} \end{bmatrix}$, the extractor outputs some $\boldsymbol{D}$ of norm $\|\boldsymbol{D}\|_\infty \le 2B$ satisfying $\boldsymbol{D} \cdot \begin{bmatrix} \boldsymbol{A} & \boldsymbol{u} & \boldsymbol{As} \end{bmatrix} = \begin{bmatrix} \boldsymbol{H} & \boldsymbol{v} & \boldsymbol{w} \end{bmatrix}$, which implies $\boldsymbol{DA} = \boldsymbol{H}$, $\boldsymbol{Du} = \boldsymbol{v}$, and $\boldsymbol{DAs} = \boldsymbol{w}$. Thus, we have $\boldsymbol{v} = \boldsymbol{Du}$ and $\boldsymbol{w} = \boldsymbol{DAs} = \boldsymbol{Hs}$, which guarantees the server's honest response with respect to $\boldsymbol{D}$. Note that such $\boldsymbol{D}$ is *computationally unique*, since the difference $(\boldsymbol{D}_1 - \boldsymbol{D}_2)$ of two such matrices $\boldsymbol{D}_1$ and $\boldsymbol{D}_2$ would yield a solution to $(n, m, q, 4B)$-SIS on $\boldsymbol{A}^T$. The underlying LHE scheme should support all the possibilities of extracted matrices to prevent the malicious server from learning information due to decryption failures. □

6. Refer to Lemma A.2 for the correctness condition for underlying LHE.

Remark 3.8 (Black-box Usage of FHE)**.** A key feature of our protocol is that it proves and verifies the honest execution of homomorphic computation, while it accesses FHE only in a black-box manner. In other words, it is agnostic to the specifics of the underlying FHE scheme or the details of how we perform matrix multiplication under FHE. In particular, the verification relies almost entirely on the decrypted result $\boldsymbol{w}$ and avoids direct interaction with the FHE ciphertexts. This is crucial for ensuring that the entire protocol remains concretely efficient. Notably, despite the inherent complexity of FHE, the efficiency of the new verification step is comparable to that of the VeriSimplePIR [24].

Remark 3.9 ((In)Dependence on $\boldsymbol{H}$)**.** At this point, it is not clear why basing vPIR on hintless PIR schemes is useful at all, as while we do not use $\boldsymbol{H}$ in the decryption but we still need $\boldsymbol{H}$ in the verification anyway. However, as will become clear later (Sections 3.3 and 3.4), this approach enables other techniques to reduce the client's online state significantly. Specifically, we will eliminate $\boldsymbol{H}$ entirely from the online phase by splitting the verification process into two parts: the first part, which uses the hint $\boldsymbol{H}$, is executed in the input-independent offline phase, while the second part, run in the online phase, does not rely on $\boldsymbol{H}$ at all (§3.3 and Remark 3.13). To achieve this, it is crucial that the second part is entirely independent of $\boldsymbol{H}$, which we accomplish by interpreting $\boldsymbol{Hs}$ as $\boldsymbol{D}(\boldsymbol{As})$.

3.3. Reusable Proof with Simpler Preprocessing

In the vLHE scheme of §3.2, the proof $\boldsymbol{Z}$ is computed for each query. While this may seem natural and inevitable, VeriSimplePIR [24] introduced a framework where a single *preprocessed* proof can be used to verify multiple queries. Their key insight was that the proof $\boldsymbol{Z} = \boldsymbol{CD}$ is entirely independent of the query and response, except for the fact that the random challenge $\boldsymbol{C}$ is derived from them for Fiat-Shamir transform. If we forgo the Fiat-Shamir transform and instead allow the client to sample the challenge $\boldsymbol{C} \xleftarrow{\$} \{0,1\}^{\kappa \times \ell}$ and send it to the server, the proof can be preprocessed during the offline phase. However, indeed, if the server learns the challenge $\boldsymbol{C}$ before the online phase, it could potentially act maliciously, using this knowledge to its advantage. To address this issue, the authors of VeriSimplePIR applied their own verifiable LHE recursively, enabling the server to compute the hint without knowing the challenge $\boldsymbol{C}$. However, this introduced complexities and inefficiencies. For instance, their approach requires an

additional commitment to $\boldsymbol{D}^T$ and a verification procedure to ensure that the commitments to $\boldsymbol{D}$ and $\boldsymbol{D}^T$ correctly correspond to a matrix and its transpose.

Simpler Preprocessing. We build on VeriSimplePIR's approach of preprocessing reusable proofs but introduce a significantly simpler and more efficient transformation. Our key insight is that verifiable LHE is not actually required for preprocessing; any (non-verifiable) LHE scheme suffices. Notably, our preprocessing eliminates the need for a second commitment on $\boldsymbol{D}^T$. For more detailed comparison with VeriSimplePIR, refer to Remarks 3.12 and 3.13. Our full protocol is detailed in Figure 5, along with its security proof in Theorem 3.11. Before we prove security, we state and our prove our main lemma.

Lemma 3.10. *(Main lemma). In Figure 5, if the server can pass the offline phase with non-negligible probability in κ, and if all the checks in Figure 5 pass, the following holds with overwhelming probability if $(n, m, q, (2\ell+1)\cdot B)$-SIS is hard:*

- *$\exists$ an efficiently extractable $\tilde{\boldsymbol{D}} \in \mathbb{Z}^{\ell \times m}$ such that $\|\tilde{\boldsymbol{D}}\|_\infty \leq 2B$ and $\boldsymbol{H} = \tilde{\boldsymbol{D}} \cdot \boldsymbol{A}$,*
- *$\boldsymbol{v} = \tilde{\boldsymbol{D}} \cdot \boldsymbol{u} \bmod q$, and*
- *$\boldsymbol{w} = \boldsymbol{H} \cdot \boldsymbol{s} \bmod q$.*

Proof. If the server passes the offline phase with non-negligible probability, we can construct an extractor $\mathcal{E}$ (Lemma 2.4) that outputs some $\tilde{\boldsymbol{D}}$ via rewinding s.t. $\|\tilde{\boldsymbol{D}}\|_\infty \leq 2B$ and $\tilde{\boldsymbol{D}} \cdot \boldsymbol{A} = \boldsymbol{H}$. Since the check passes, we have $\boldsymbol{Z} \cdot \boldsymbol{A} = \boldsymbol{C} \cdot \boldsymbol{H} = \boldsymbol{C} \cdot (\tilde{\boldsymbol{D}} \cdot \boldsymbol{A})$. Rearranging the terms, we get $(\boldsymbol{Z} - \boldsymbol{C} \cdot \tilde{\boldsymbol{D}}) \cdot \boldsymbol{A} = 0$. Since $\|\boldsymbol{Z}\|_\infty \leq B$ (ensured by the checks) and $\|\boldsymbol{C} \cdot \tilde{\boldsymbol{D}}\|_\infty \leq 2\ell B$, we have that: $\|\boldsymbol{Z} - \boldsymbol{C} \cdot \tilde{\boldsymbol{D}}\|_\infty \leq 2\ell B + B = (2\ell+1) \cdot B$. Assuming that (n, m, q, β)-SIS problem is hard for $\beta = (2\ell+1) \cdot B$, we have that $\boldsymbol{Z} = \boldsymbol{C} \cdot \tilde{\boldsymbol{D}}$; otherwise we get an SIS solution for $\boldsymbol{A}^T$. Lastly, in the online phase, the checks performed are: $\boldsymbol{Z} \cdot [\boldsymbol{u} \quad \boldsymbol{A} \cdot \boldsymbol{s}] = \boldsymbol{C} \cdot [\boldsymbol{v} \quad \boldsymbol{w}]$. Replacing $\boldsymbol{Z} = \boldsymbol{C} \cdot \tilde{\boldsymbol{D}}$, we get: $(\boldsymbol{C} \cdot \tilde{\boldsymbol{D}}) \cdot [\boldsymbol{u} \quad \boldsymbol{A} \cdot \boldsymbol{s}] = \boldsymbol{C} \cdot [\boldsymbol{v} \quad \boldsymbol{w}]$. Therefore, we have $\boldsymbol{C} \cdot (\tilde{\boldsymbol{D}}\boldsymbol{u} - \boldsymbol{v}) = 0$ and $\boldsymbol{C} \cdot (\tilde{\boldsymbol{D}}\boldsymbol{A}\boldsymbol{s} - \boldsymbol{w}) = 0$. Since $\boldsymbol{C}$ is hidden from the server (w.h.p. nothing about $\boldsymbol{C}$ leaked if $\boldsymbol{Z}\boldsymbol{A} = \boldsymbol{C}\boldsymbol{H}$ passes because it implies $\boldsymbol{Z}$ computed honestly w.h.p.) and drawn from $\{0,1\}^{\kappa \times \ell}$[7], using Lemma A.1, we get $\tilde{\boldsymbol{D}}\boldsymbol{u} = \boldsymbol{v}$ and $\tilde{\boldsymbol{D}}\boldsymbol{A}\boldsymbol{s} = \boldsymbol{w}$ with overwhelming probability. Given $\tilde{\boldsymbol{D}}\boldsymbol{A} = \boldsymbol{H}$ (proved before), we get $\boldsymbol{H}\boldsymbol{s} = \boldsymbol{w}$. □

Theorem 3.11. *(Our protocol is vLHE). Figure 5 is a vLHE scheme (Definition 3.4) as long as $(n, m, q, (2\ell+1)\cdot B)$-SIS is hard[8] and LHE correctness[9] holds with overwhelming probability for linear transformations of ∞-norm $\leq 2B$.*

Proof. We provide a proof sketch here, and defer the full proof to §A.2. We construct a simulator $\mathcal{S}$ that runs $\mathcal{E}$ (Lemma 2.4) to extract an id-specific database $\boldsymbol{D}_{\mathsf{id}}$ from $\mathcal{A}$ when a register request comes in, and sends this to $\mathcal{F}$. The functionality uses $\boldsymbol{D}_{\mathsf{id}}$ to respond to all queries with a matching id. For each query to $\mathcal{F}$, $\mathcal{S}$ sends a dummy query to $\mathcal{A}$. Semantic security of encryption ensures the view of $\mathcal{A}$ is indistinguishable from a real protocol. From Lemma 3.10, if $\mathcal{A}$ cheats, the simulator detects it and sends abort signal to $\mathcal{F}$, similar to the client aborting in the real execution. □

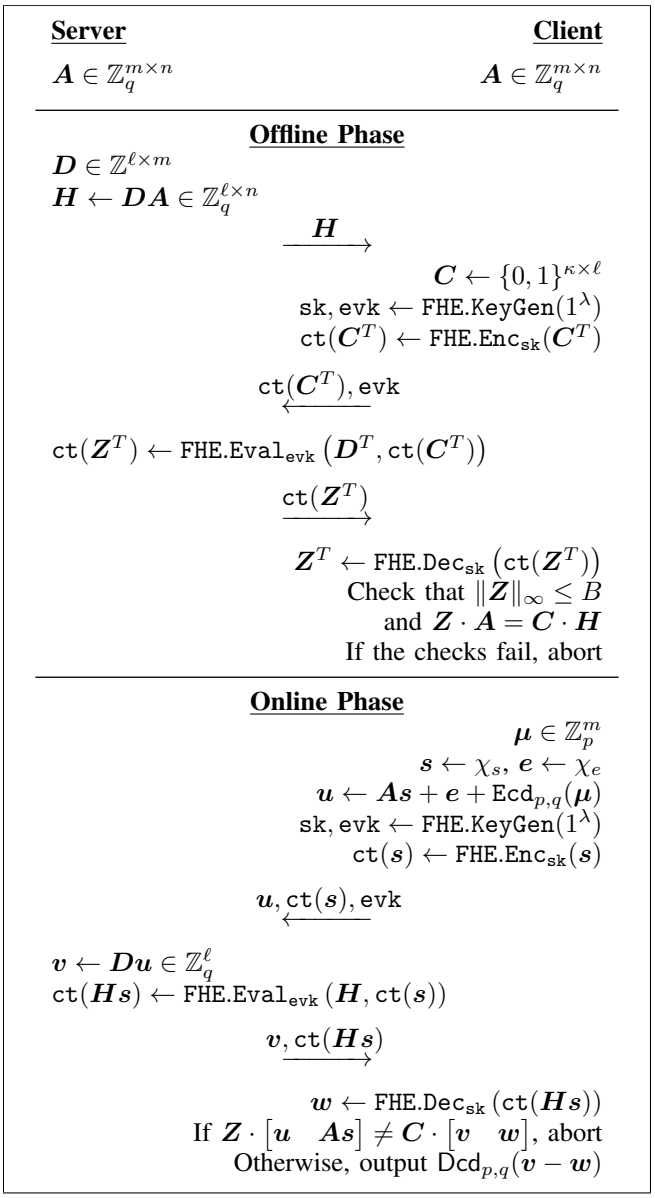

Figure 5: vLHE from Hintless PIR with Reusable Proof. B is described in Figure 2.

Remark 3.12 (Comparison to VeriSimplePIR (1): Offline Phase)**.** A key distinction between our offline phase and that of VeriSimplePIR (for details, see the original paper [24, Fig. 6]) is that our approach can be instantiated with any

7. $\boldsymbol{C}$ is independent of $\tilde{\boldsymbol{D}}$; the extractor is run on a set of challenges to extract $\tilde{\boldsymbol{D}}$ such that $\tilde{\boldsymbol{D}} \cdot \boldsymbol{A} = \boldsymbol{H}$, and a fresh $\boldsymbol{C}$ is then sampled to compute $\boldsymbol{Z}$ used in online phase.

8. When $\boldsymbol{H}$ is honestly generated using a database $\boldsymbol{D}$, this can be reduced to $(n, m, q, (B + \ell \cdot \|\boldsymbol{D}\|_\infty))$-SIS.

9. Please refer to Lemma A.2 for details on the correctness condition for underlying LHE.

LHE, whereas VeriSimplePIR specifically requires a vLHE. To achieve an efficient instantiation, VeriSimplePIR recursively employs its own vLHE within the offline phase. However, this approach introduces several complexities and inefficiencies in VeriSimplePIR: (1) an additional commitment to $\boldsymbol{D}^T$ is required alongside the commitment to $\boldsymbol{D}$, (2) an extra verification step is needed to ensure consistency between these commitments for the matrix and its transpose, and (3) the Fiat-Shamir heuristic is employed to minimize interaction, similar to the approach in Figure 4. In contrast, our simpler preprocessing avoids all of these drawbacks, as shown in Figure 5. Specifically, our approach uses only a single commitment to $\boldsymbol{D}$ and does not rely on the Fiat-Shamir heuristic. The trade-off is that we require a slightly larger parameter for SIS hardness: the bound parameter β is $(2\ell+1)\cdot B$ in our scheme (Theorem 3.11) and $\approx 4B$ in VeriSimplePIR. This is due to a new bound on $\boldsymbol{Z}$-related terms that exceeds the bound for $\boldsymbol{D}$. However, this has a negligible impact on performance under practical parameters (see §6).

Remark 3.13 (Comparison to VeriSimplePIR (2): Online Phase)**.** Although our simpler preprocessing primarily targets the offline phase, when combined with our *hintless* approach from §3.2, it also enhances the online phase compared to VeriSimplePIR. Notably, as promised in Remark 3.9, clients no longer require access to the hint $\boldsymbol{H} \in \mathbb{Z}_q^{\ell \times n}$ during the online phase. While this provides a considerable reduction in the online state, however, clients still need to maintain $\boldsymbol{Z} \in \mathbb{Z}^{\kappa \times m}$ such that $\|\boldsymbol{Z}\|_\infty \leq B$ and $\boldsymbol{C} \in \{0,1\}^{\kappa \times \ell}$. We further reduce the required persistent storage of the clients in §3.4.

3.4. Compressing Online State

We now present our approach for compressing the preprocessed proof $\boldsymbol{Z}$. As discussed in §3.3, the proof $\boldsymbol{Z} \in \mathbb{Z}^{\kappa \times m}$, where $\|\boldsymbol{Z}\|_\infty \leq B$, constitutes the majority of the long-term online state that the client must maintain after the one-time preprocessing phase. At a high level, our core idea is to compress the proof by having the client take random linear combinations of the rows of $\boldsymbol{Z}$, effectively substituting the original $\boldsymbol{Z}$ with its compressed representation. More precisely, the following modifications are made to the client-side of the protocol. We note that the ciphertext modulus q is set to be a prime number so that $\mathbb{Z}_q$ is a field. (See Remark 3.15 for further discussion.)

- **Offline Phase:** Set $\gamma := \lceil \kappa / \log q \rceil$. At the very end of the offline phase, instead of checking $\boldsymbol{Z}\cdot\boldsymbol{A} = \boldsymbol{C}\cdot\boldsymbol{H}$, the client samples $\boldsymbol{R} \xleftarrow{\$} \mathbb{Z}_q^{\gamma \times \kappa}$. The client then computes and stores $\boldsymbol{Z}' \leftarrow \boldsymbol{R}\cdot\boldsymbol{Z} \in \mathbb{Z}_q^{\gamma \times m}$ and $\boldsymbol{C}' \leftarrow \boldsymbol{R}\cdot\boldsymbol{C} \in \mathbb{Z}_q^{\gamma \times \ell}$. Finally, the client checks that $\boldsymbol{Z}'\cdot\boldsymbol{A} = \boldsymbol{C}'\cdot\boldsymbol{H}$. Note that the client can discard $\boldsymbol{Z}$ and $\boldsymbol{C}$, as they are no longer needed for the online phase.
- **Online Phase:** The client's verification step $\boldsymbol{Z}\cdot\begin{bmatrix}\boldsymbol{u} & \boldsymbol{As}\end{bmatrix} = \boldsymbol{C}\cdot\begin{bmatrix}\boldsymbol{v} & \boldsymbol{w}\end{bmatrix}$ is replaced with $\boldsymbol{Z}'\cdot\begin{bmatrix}\boldsymbol{u} & \boldsymbol{As}\end{bmatrix} = \boldsymbol{C}'\cdot\begin{bmatrix}\boldsymbol{v} & \boldsymbol{w}\end{bmatrix}$.

Remark 3.14 (Improvement from Compression)**.** Our proof compression replaces $\boldsymbol{Z} \in \mathbb{Z}^{\kappa \times m}$ such that $\|\boldsymbol{Z}\|_\infty \leq B$ (and $\boldsymbol{C} \in \{0,1\}^{\kappa \times \ell}$) with $\boldsymbol{Z}' \in \mathbb{Z}_q^{\gamma \times m}$ and $\boldsymbol{C}' \in \mathbb{Z}_q^{\gamma \times \ell}$. Therefore, if we set $B = \ell \cdot p$, we achieve a reduction in proof-related state from $\kappa \cdot m \cdot \log(\ell \cdot p)$ bits to $\gamma \cdot (m+\ell) \cdot \log q$ bits. For instance, under our parameters for an 8 GiB database (§6), compression alone reduces the state size from 8000 KiB to 1536 KiB, achieving a $5.2\times$ improvement.

Remark 3.15 (Prime Field vs Power-of-Two Ring)**.** In our vLHE scheme, we set the ciphertext modulus q to be a prime number rather than a power of two (2^{32} or 2^{64}) used in prior works (VeriSimplePIR [24], HintlessPIR [51]). This choice not only simplifies the procedure and analysis of the compression technique described above but also allows us to align q with the native plaintext modulus of the BFV scheme [14], [30] used in our online phase to instantiate an FHE scheme. Therefore, we can use better parameters for delegation, leading to improved communication and computation. One potential drawback of using a non-power-of-two q is that the server cannot directly leverage machine-native arithmetic over 2^{32} or 2^{64} when computing the matrix multiplication $\boldsymbol{Du}$ in the online phase. We implement careful optimizations (such as using a Solinas prime q for faster modular arithmetic and employing lazy modular reduction) to reduce this overhead, and observe that a prime q is a more balanced all-around choice. See also §6.

4. Covert vLHE

We now turn to the security of *stateless* clients, which—as discussed in §1, are motivated by real-world scenarios involving infrequent users. The definition of vLHE (Definition 3.4) doesn't provide any security guarantees to stateless clients. Therefore, all that a stateless client is guaranteed is the (semi-honest) standard PIR security, i.e. the client's query doesn't reveal any private information, but the server's response can be arbitrary allowing it to mount selective failure attack against the client and learning private information. In this section, we introduce the notion of Covert vLHE (cvLHE) which offers stronger security to stateless clients.

4.1. Definition and Construction of cvLHE

Consider a system where stateful and stateless clients coexist, and the server doesn't know aprori if the next request will be from a stateful or a stateless client; assume that the requests are stripped of source IP address by an anonymizing proxy. In this system, a stateless client can "pretend" to be stateful by locally simulating fake state and behaving like a stateful client in an attempt to fool the server into believing that the client can check if the server alters the response. However, the definition of vLHE doesn't mandate any structure from the requests of stateful clients. Therefore, the server might be able to identify faking (stateless) clients from the requests.

Property 1: Stateful-Stateless Indistinguishability. We address this problem by observing that our vLHE scheme

(Figure 5) already has an important property - online queries made by the clients are independent of their persistent state. Therefore, a stateless client can make an identical looking query to a stateful client, and the server cannot identify if a request is stateful or stateless. This gives stateless clients the power to hide among stateful clients. While this already improves the security of stateless clients, this doesn't establish a strong deterrent for the server to not cheat. The server has a lot to gain by cheating in responding to a given query. On one hand if the query was stateless, then the server successfully mounts selective failure, and on the other hand, even if it was stateful and the server is caught, the evidence is private to a single client—meaning the server faces no long-term reputational or operational consequences and can continue cheating with other clients.

Property 2: Publicly-Verifiable Stateful Online Cheating. A crucial deterrent against server misbehavior is ensuring that, if the server is ever caught cheating by a stateful client in responding to a query, the violation can be publicly verified—making even a single exposure reputationally costly. To achieve this property, we have to make some changes to our vLHE scheme. We first formally define cvLHE and then discuss the required changes to our protocol. In §A.3, we further extend cvLHE to a stronger notion - Strong cvLHE.

Definition 4.1 (Stateful and Stateless clients)**.** *In an offline-online client-server protocol (Definition 3.1), define:*

- Stateful clients*: Execute both the offline and online protocols with the server.*
- Stateless clients*: Execute only the online protocol with the server; ignore all checks and corresponding aborts.*

Definition 4.2 (Stateful-Stateless Indistinguishability Game STATE-IND)**.** *Consider a game between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$. $\mathcal{C}$ acts as a stateful client and runs the offline protocol π_0 with $\mathcal{A}$. If $\mathcal{C}$ aborts during π_0, $\mathcal{A}$ loses. Denote $\mathcal{C}$'s state after π_0 as $\mathtt{st}$. $\mathcal{C}$ flips a coin $b \in \{0,1\}$ and sets $\mathtt{st} \leftarrow \{\}$ if $b = 1$. Then, $\mathcal{C}$ executes the online protocol π_1[10], with state $\mathtt{st}$, as a stateful client if $b = 0$, and a stateless client otherwise. $\mathcal{A}$ wins if it correctly guesses b.*

Definition 4.3 (cvLHE)**.** *A cvLHE scheme is an offline-online client-server protocol (Definition 3.1) with stateful and stateless clients (Definition 4.1). It satisfies the following properties*

- Universal LHE*: Correctness and semantic security of LHE §2.2 to all clients.*
- Stateful vLHE*: vLHE to stateful clients (Definition 3.4).*
- Stateful-Stateless Indistinguishability*: For all PPT $\mathcal{A}$, the winning probability in* STATE-IND *(Definition 4.2) is at most $\frac{1}{2}+\mathsf{ngl}(\lambda)$, where λ is the security parameter.*
- Publicly-Verifiable Stateful Online Cheating*: For a disputed stateful client, given the online-protocol transcript τ, linear transformation $\boldsymbol{D}$ from the server, and server-signed commitment h of the server's linear transformation from the client's input-independent offline phase transcript, $\exists$ a cheater identification algorithm* ConfirmCheater *such that if the server's response in τ is inconsistent w.r.t the client's query in τ and linear transformation committed by h, then w.h.p.* ConfirmCheater *outputs* 1*. Otherwise, outputs* 0*. The client's private query is never revealed.*

Construction 4.4 (Our cvLHE scheme)**.** *We describe the changes to our vLHE scheme (Figure 5) to construct a cvLHE scheme. We assume a PKI that stores the public key of the server. We don't protect against denial-of-service (DoS) attacks by the server.*

- Offline Phase*: In the first message from the server to the client, it attaches a signed collision-resistant (CR) hash h of the hint $\boldsymbol{H}$. The client aborts if the $\boldsymbol{H}$ it receives is inconsistent with h; this abort is equivalent to the server denying service.*
- Online Phase*: Since queries need to be anonymous, we can't use client-signed queries. Instead, clients make unsigned queries and the server responds with signed query-response pair. If the server alters the query under its signature, it is equivalent to simply denying service to the client.*

Efficiency. Our cvLHE scheme preserves the online latency, server throughput, and offline efficiency of our vLHE protocol (Figure 5; §6). Offline phase has an extra λ-bit h, and online phase has a new signature on the query-response pair.

Theorem 4.5. *Construction 4.4 is a cvLHE scheme given that Figure 5 is a vLHE scheme.*

Proof. Universal LHE and Stateful vLHE hold by construction. For Stateful-Stateless Indistinguishability, observe that in Figure 5, the online query of the client $(\boldsymbol{u}, \mathtt{ct}(\boldsymbol{s}), \mathtt{evk})$ is independent of the state $\mathtt{st} = (\boldsymbol{Z}, \boldsymbol{C})$ output by offline phase—the online query of a stateful and stateless client are identical. Therefore, the best that $\mathcal{A}$ can do to guess the bit b is to sample a uniform coin. For Publicly-Verifiable Stateful Online Cheating, ConfirmCheater works as follows. It verifies the server's signature on h. If verification fails, then output 0—either the client is cheating or DoS by server. Then, it computes $\boldsymbol{H} \leftarrow \boldsymbol{D} \cdot \boldsymbol{A}$ ($\boldsymbol{D}$ provided by the server and $\boldsymbol{A}$ is in public parameters) and verifies that its CR hash matches h. If this fails, output 1 (server cheated). Note that by the hardness of SIS, $\boldsymbol{H}$ is a binding commitment. Next, it verifies the signature on query-response pair in online transcript τ. If signature verification fails, output 0 (cheating client or DoS). At last, it verifies that the response is correctly computed given the query (from the server-signed query-response pair), $\boldsymbol{D}$, and $\boldsymbol{H}$—the response is deterministic given these quantities. If verification fails, output 1. Otherwise, output 0. Note that the client's outgoing query to the server in τ could be different than the query that the server signs in query-response pair. This is equivalent to DoS and the client aborts when this happens. Therefore, such transcripts won't be disputed by the client. □

Outside the scope of cvLHE. We only focus on the cryptographic aspect of the security of stateless clients. Attacks

10. Note that aborts are not communicated to $\mathcal{A}$ in online protocol π_1.

exploiting the request pattern of clients (e.g. time of day they are active) are beyond our scope.

4.2. Practical Considerations for cvLHE

We now discuss three important considerations for the adoption of cvLHE. First, if malicious clients mount a DDoS attack against the server, it can be costly in terms of depleting the server's compute resources. We address this concern via anonymous rate limiting. Second, the security of stateless clients relies on there being enough stateful clients to deter the server from ever cheating. This raises two questions: (1) what fraction of clients should be stateful, and (2) how to achieve this fraction. To answer these questions, we investigate the game theory behind cvLHE. Third, enabling efficient database updates in cvLHE can be important for certain real-world applications. We discuss an update-friendly deployment setup that relies on designated auditors.

Anonymous rate limiting. It is straightforward to use anonymous credentials like PrivacyPass [59] with our protocol to enforce rate limiting while preserving anonymity and preventing DDoS attacks. Using PrivacyPass, clients first authenticate conventionally to a service that blindly issues access tokens; anonymity is not required here. Later, the client attaches an unused token with each online anonymous query. PrivacyPass ensures that the server cannot link token redemption to issuance, preserving the client's anonymity.

Reaching equilibrium state. In §A.4, we provide an initial game-theoretic analysis for cvLHE via an asymmetric pure equilibrium, where the server is honest, some clients are stateful, and the rest are stateless. To reach this equilibrium state, our goal is to create a critical mass of stateful clients; this mass is relatively small as discussed in §A.4. To do this, at initialization, the server runs the setup with this critical mass of stateful clients; we assume the application developer starts with a fresh honest server during initialization. This ensures that a malicious server cannot lie to the clients about the number of stateful clients in the system to make them believe that the critical mass has reached, when in reality it hasn't; recall that the server knows exactly how many stateful clients are there because it receives that many registration requests. Only after the mass is reached does the server respond to any online-phase queries. The equilibrium says that from this point onwards, the server's optimal strategy is to be honest if stateful mass doesn't degrade. Note that a stateful client wouldn't switch to being stateless under this equilibrium because doing so would make the server's optimal strategy to cheat, and that harms this (earlier stateful; now stateless) client. Therefore, the system now operates freely without worrying about a (rational) compromised server. To incentivize clients to participate in this initialization, we can reward them with anonymous access tokens (e.g. PrivacyPass). Moreover, these designated stateful clients could also be auditors run by independent organizations. We leave an in-depth game-theoretic analysis to future work.

Update-friendly setup with auditors. Real-world databases evolve over time and for this, it is important that clients are stateless to prevent a network-wide communication blowup whenever a new update is pushed to the database, especially when the preprocessing communication is large; note that preprocessing is required on each update. While building a concretely efficient vPIR scheme with small preprocessing communication remains an open problem, we can use auditors to remedy this issue in cvLHE. Auditors run by independent parties can regularly issue requests to the server, verify that it is not cheating, and do preprocessing on database updates, while all the clients can be fully stateless. Then cvLHE guarantees that as long as we have sufficiently many auditor requests, the server is deterred from cheating for *all* requests. Importantly, auditors can be significantly cheaper to run than the server because an auditor is simply a stateful client.

5. Optimizations

Query-level preprocessing. Preprocessing in PIR can be divided into three categories. The highest is *database-level preprocessing* that is performed exclusively by the server (e.g. computing $\boldsymbol{H} = \boldsymbol{DA}$). Next, there is *client-level preprocessing* that is performed once per client (e.g. precomputation of reusable proof (§3.3) in our vLHE scheme). The last one is *query-level preprocessing* which generates a consumable (and non-reusable) state for the online phase.

To improve the online latency of our protocol, we propose leveraging query-level preprocessing. This approach was not utilized in our direct predecessors: SimplePIR [41], VeriSimplePIR [24], and HintlessPIR [51]. We observe that many computationally expensive parts of our vLHE scheme (Figure 5) are independent of the client's input and can therefore be preprocessed. This includes the expansion of $\boldsymbol{A}$ from its seed and the secure delegation of $\boldsymbol{Hs}$ to the server via FHE. The storage overhead on the client from this additional state stays low; under a MiB for an 8 GiB database. Therefore, the client can even preprocess a batch of queries together to later execute an entire *online session* at boosted latency.

Remark 5.1 (Reusing proof after verification failure)**.** The authors of VeriSimplePIR [24] required a rerun of client-level proof preprocessing whenever proof verification failed in online phases. However, in our protocol, the client can safely reuse the same proof up to $\mathsf{poly}(\lambda, \kappa)$ times, regardless of the verification output in online phase. This is because the server can simulate the client's abort w.h.p., which ensures that only negligible information is leaked to the server[11]. Notably, this argument also applies to the original VeriSimplePIR protocol.

BSGS-style homomorphic matrix multiplication. Recall that proof preprocessing in our vLHE scheme (Figure 5) is

11. This requires a stronger version of Lemma A.1 which holds when $\boldsymbol{C}$ is close to uniform in statistical distance. It directly follows from the fact that the statistical distance between the distributions $\boldsymbol{C} \cdot \boldsymbol{x}$, where $\boldsymbol{C}$ is uniform versus δ-close to uniform, is bounded by δ.

agnostic to the choice of underlying FHE scheme or how the server computes $\boldsymbol{H}\boldsymbol{s}$. To enhance efficiency of this step, we employ Baby-Step Giant-Step (BSGS)-style homomorphic matrix multiplication [39]. For the sizes of databases we consider, this results in fewer or equal homomorphic rotations to HintlessPIR. While this technique slightly increases the upload size due to one additional rotation key (Horner-style [39]), compared to HintlessPIR, it is not noticeable relative to the total communication cost. In fact, the BSGS optimization enables tighter packing (see §6.1), which *reduces* total communication (see §6.2).

Streaming-friendly offline phase. So far we have only focused on reducing the client's online state. To reduce the client's offline state, we rely on streaming. The idea is to stream columns of $\boldsymbol{H}$ to the client, one at a time. This has to be done only once since this is a one-time phase. The client computes and stores $\boldsymbol{Z}' \leftarrow \boldsymbol{R} \cdot \boldsymbol{Z}$ and $\boldsymbol{C}' \leftarrow \boldsymbol{R} \cdot \boldsymbol{C}$ in a streaming fashion (sending encryption of one row of $\boldsymbol{C}$ to the server at a time). Then, the server streams $\boldsymbol{H}$ to the client column-wise and the client performs the check $\boldsymbol{Z}' \cdot \boldsymbol{A} = \boldsymbol{C}' \cdot \boldsymbol{H}$ column-wise. Note that all of the client's local computation in offline phase is friendly to streaming.

6. Implementation and Evaluation

6.1. Implementation

We implement our protocol in C++, building over the codebase of HintlessPIR [37] and VeriSimplePIR [15]. For query-level preprocessing, we instantiate our scheme with BFV (Brakerski/Fan-Vercauteren) FHE scheme [14], [30], which is based on the hardness of RLWE [53]. We perform homomorphic matrix multiplication in BSGS-style (§5). For client-level preprocessing, we use the LHE scheme from SimplePIR (§2.4). Our implementation is single-threaded. Similar to HintlessPIR, whenever supported by the CPU, our code uses AVX-512 SIMD instruction set for all matrix multiplications involving the database $\boldsymbol{D}$ via Google's Highway library [36]. Our code is available here[12].

Optimizations and assumptions. We implement the database-level preprocessing optimization [41], [51] to preprocesses expensive computations on (R)LWE ciphertexts, and extend it to our BSGS-based homomorphic matrix multiplication. In our implementation and the baselines, we compress the random matrix $\boldsymbol{A}$ with a seed [41] (part of public parameters), and expand $\boldsymbol{A}$ and store $\boldsymbol{A}\boldsymbol{s}$ (significantly smaller than $\boldsymbol{A}$) in a query-level preprocessing phase to improve online latency. Our implementation assumes that the hint $\boldsymbol{H}$ is honestly generated, as is commonly considered in prior work on vPIR [17], [24], [66]. Since the hint is the same for all clients, they can check if the hint has sufficiently-many signatures of trusted parties to be confident that it is honestly generated. This allows us to use better parameters.

Parameters. We use Lattice Estimator [3], [28] (Commit ID: `162c505`) to choose (R)LWE and SIS parameters to match $\lambda \geq 128$ bits of security, and set statistical security parameter $\kappa = 40$. We set our LWE plaintext modulus $p = 2^8$ and all database entries are split into 1 byte chunks. Unlike prior works [24], [51] which use power-of-two values for q (e.g., 2^{32}, 2^{64}), we set the LWE ciphertext modulus to $q = 2^{32} - 2^{24} + 2^{16} + 1$, a Solinas prime that is also NTT-friendly (see Remark 3.15). We sample error from centered Binomial distribution [5], [51] with variance = 8, and use uniform ternary distribution [13], [35], [51] for LWE secret key. We set LWE dimension $n = 1408$. For RLWE, we set the ring dimension 2^{12}, and sample error and secret key from centered Binomial distribution with variance 8. The RLWE plaintext modulus is set to match the LWE ciphertext modulus. We use a pair of 56-bit NTT-friendly primes for RLWE ciphertext space. Since our client-level preprocessing has to support plaintext values up to $p\ell$, we use a larger instance of LWE for it. We set the LWE dimension as 2816, plaintext modulus as $p\ell$, and ciphertext modulus as a 54-bit Solinas prime $2^{54} - 2^{24} + 1$.

Improved packing for FHE. Under our RLWE parameters, we have 2^{12} data slots in each BFV plaintext. These slots are equally laid out across two rows supporting column-wise rotation. Each row has 2^{11} slots, large enough to encode the entire LWE secret key ($n = 1408$); we pad the rest of the slots with zeros. Similar to HintlessPIR, we pack two copies of the LWE secret key $\boldsymbol{s}$, one in each row. To pack the matrix $\boldsymbol{H}$ into multiple polynomials, we first pad $\boldsymbol{H}$ with columns of zeros to get a total of 2^{11} columns. We then consider each $2^{11} \times 2^{11}$ square block of $\boldsymbol{H}$, which has 2^{11} diagonals, each of size 2^{11}. Pairs of diagonals are packed into each plaintext polynomial, with one diagonal per row. As noted by HintlessPIR, packing in both rows reduces the number of rotations and homomorphic multiplications by $2\times$ over naive packing. Moreover, since we use maximum-size (square) blocks, our packing reduces the ciphertext count in server's response by $2\times$ compared to HintlessPIR. Even though a larger block size typically increases rotations, our BSGS optimization reduces the number of rotations so significantly that we can optimize for communication.

6.2. Evaluation

Setup. All the experiments are run on `c3-highmem-44` instance of Google Cloud. This machine has 352 GB memory and Intel Sapphire Rapids processor which supports AVX-512. Communication latency is estimated over 64 Mbps network. We evaluate on databases of varying sizes (256 MiB to 16 GiB) both in the number of entries (2^{18} to 2^{34}) and the size of each entry (1 byte to 32 KiB). Due to space limitation, we defer the details on each database to §A.5.

Client's persistent and online state. Our vPIR scheme reduces persistent state over VeriSimplePIR by 1500-$2300\times$ (Figure 6). Even in terms of short-term state—the local state when making a query including query-level preprocessing—the improvement is up to $133\times$ for our largest database; it is

12. https://github.com/mayank0403/Verifiable-Hintless-PIR

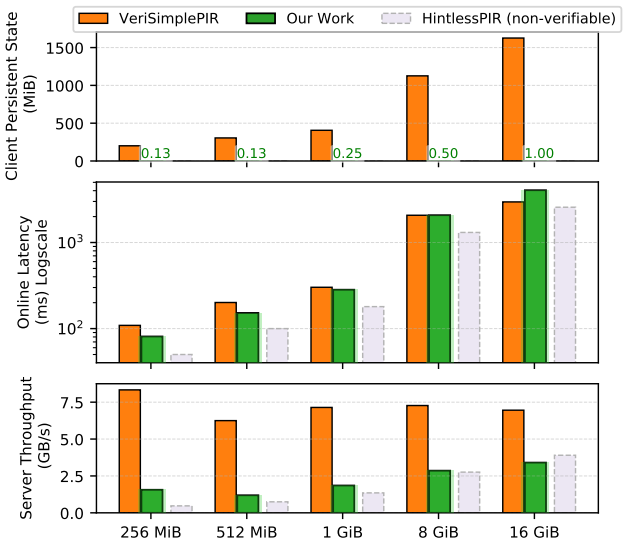

Figure 6: Comparison of our scheme with VeriSimplePIR. HintlessPIR is incomparable given its weaker security; we only show it here because our work builds over it. Persistent state is zero in HintlessPIR. For small DBs, VeriSimplePIR's latency is affected by Gaussian sampling. Throughput of our work and HintlessPIR rises with bigger DBs; cost of FHE components gets overshadowed by LHE.

at most 12 MiB. The online state of our scheme[13] is about 4 MiB for 8 GiB and 16 GiB databases, and less than a MiB for smaller databases.

Online latency. Out of the box, VeriSimplePIR (resp. HintlessPIR) has 6.4-35× (resp. 4.5-32×) higher latency than our scheme because they don't leverage query-level preprocessing. When we add this optimization to their implementation, we observe comparable online latency between our work and VeriSimplePIR (Figure 6). The gap is widest for a 16 GiB database, where our online latency is about 1.4× higher than VeriSimplePIR (1.6× over HintlessPIR). This captures the impact of having to use a non-power-of-two q in our scheme (Remark 3.15).

Server throughput[14]. We report the server throughput comparison in Figure 6. As shown above, our vPIR scheme improves the client storage by orders of magnitude, but this comes at an overhead in throughput. In fact, this trade-off exists even beyond our scheme and vPIR - state-of-the-art non-verifiable hint-free schemes like HintlessPIR and YPIR show a similar behavior. For large databases (when linear LHE cost dominates over sublinear FHE), the overhead in throughput incurred by our vPIR scheme is about 2-2.5× over VeriSimplePIR. This can be reduced to as low as 1.8× by arranging DB as a wide matrix as we discuss in §A.5.

13. After removing preprocessing comm. (Table 2) from short-term state. Includes: consumable state o/p by query-level preprocessing, $\boldsymbol{Z}'$, $\boldsymbol{C}'$, etc.

14. Throughput is a standard metric influenced by total server compute per query: includes both online and query-level preprocessing compute.

Performance breakdown and preprocessing cost. To highlight the improvements from our optimizations, we breakdown the computation and communication costs of the server and the client after one-time (database-level and client-level) preprocessing is done. With our BSGS optimization, better packing, and use of NTT-friendly prime q, we observe more than 3× improvement over HintlessPIR in terms of client download during query-level preprocessing. This reduces total communication by 2.5× over HintlessPIR. Moreover, these optimizations improve server's computation during preprocessing by 2× over HintlessPIR. We defer the detailed breakdown to Table 2 in §A.5. We also discuss the cost of one-time preprocessing in §A.5.

Trade-off and future improvements. Our vPIR scheme provides large improvements in client storage with comparable online latency to VeriSimplePIR, however, this comes at an overhead in throughput and preprocessing communication. This trade-off is inherited from HintlessPIR; we implement several optimizations to improve this. In §A.5, we point out other optimizations to further reduce preprocessing communication. Their implementation is left for future work.

7. Extensions and Future Work

Database privacy (Symmetric PIR). It is common for PIR works to assume a public database. However, in many real-world applications where the database is maintained by an organization and constitutes part of its intellectual property, preserving the privacy of the database itself becomes crucial. Symmetric PIR (SPIR) [58] targets this problem and requires privacy for both the client and the server, where for the server's privacy, the client learns nothing other than the database entry at the index they requested. In §A.6, we discuss a natural extension of our vPIR scheme to construct verifiable symmetric PIR (vSPIR).

Future work. We briefly describe open problems and directions for future work. Firstly, lowering the communication of the preprocessing phase remains an open question. In our protocol and VeriSimplePIR, this phase is dominated by the communication of the hint which can be gigabytes large. Secondly, arbitrary database updates create tension with vPIR's consistency property. If the server is allowed to change the database arbitrarily, then having database consistency between responses becomes less meaningful. Lastly, for situations where the database changes frequently, re-running the preprocessing phase with each client can be expensive (see §4.2 for discussion on an auditor setup to mitigate this issue). While infrequent or sparse updates can be handled with delta-preprocessing, it remains an open question how to efficiently support the general case. A direction for future work is to do an extensive study of the game theory behind cvLHE (see §A.4).

Acknowledgments

We thank the anonymous reviewers, the shepherd, Emma Dauterman, Alexandra Henzinger, Sanjam Garg, Bhaskar

Roberts, and Naomi Barrall for their helpful feedback. We thank Leo de Castro and Baiyu Li for helpful discussions, and Agam Gambhir for participating in the initial part of this work. We also thank students in the Sky security group for feedback that improved the presentation of this paper.

References

[1] M. Ajtai, "Generating hard instances of lattice problems (extended abstract)," in *STOC*. ACM, 1996, pp. 99–108.

[2] ——, "Generating hard instances of lattice problems (extended abstract)," in *28th Annual ACM Symposium on Theory of Computing*. Philadephia, PA, USA: ACM Press, May 22–24, 1996, pp. 99–108.

[3] M. R. Albrecht, R. Player, and S. Scott, "On the concrete hardness of learning with errors," *J. Math. Cryptol.*, vol. 9, no. 3, pp. 169–203, 2015.

[4] A. Ali, T. Lepoint, S. Patel, M. Raykova, P. Schoppmann, K. Seth, and K. Yeo, "Communication-computation trade-offs in PIR," in *USENIX Security Symposium*. USENIX Association, 2021, pp. 1811–1828.

[5] E. Alkim, L. Ducas, T. Pöppelmann, and P. Schwabe, "Post-quantum key exchange - A new hope," in *USENIX Security 2016: 25th USENIX Security Symposium*, T. Holz and S. Savage, Eds. Austin, TX, USA: USENIX Association, Aug. 10–12, 2016, pp. 327–343.

[6] S. Angel, H. Chen, K. Laine, and S. T. V. Setty, "PIR with compressed queries and amortized query processing," in *IEEE Symposium on Security and Privacy*. IEEE Computer Society, 2018, pp. 962–979.

[7] D. F. Aranha, A. Costache, A. Guimarães, and E. Soria-Vazquez, "HELIOPOLIS: verifiable computation over homomorphically encrypted data from interactive oracle proofs is practical," in *ASIACRYPT (5)*, ser. Lecture Notes in Computer Science, vol. 15488. Springer, 2024, pp. 302–334.

[8] H. Asi, F. Boemer, N. Genise, M. H. Mughees, T. Ogilvie, R. Rishi, G. N. Rothblum, K. Talwar, K. Tarbe, R. Zhu, and M. Zuliani, "Scalable private search with wally," *CoRR*, vol. abs/2406.06761, 2024.

[9] Y. Aumann and Y. Lindell, "Security against covert adversaries: Efficient protocols for realistic adversaries," in *TCC 2007: 4th Theory of Cryptography Conference*, ser. Lecture Notes in Computer Science, S. P. Vadhan, Ed., vol. 4392. Amsterdam, The Netherlands: Springer, Berlin, Heidelberg, Germany, Feb. 21–24, 2007, pp. 137–156.

[10] C. Baum, J. Bootle, A. Cerulli, R. del Pino, J. Groth, and V. Lyubashevsky, "Sub-linear lattice-based zero-knowledge arguments for arithmetic circuits," in *Advances in Cryptology – CRYPTO 2018, Part II*, ser. Lecture Notes in Computer Science, H. Shacham and A. Boldyreva, Eds., vol. 10992. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 19–23, 2018, pp. 669–699.

[11] S. Ben-David, Y. T. Kalai, and O. Paneth, "Verifiable private information retrieval," in *TCC (3)*, ser. Lecture Notes in Computer Science, vol. 13749. Springer, 2022, pp. 3–32.

[12] A. Bois, I. Cascudo, D. Fiore, and D. Kim, "Flexible and efficient verifiable computation on encrypted data," in *PKC 2021: 24th International Conference on Theory and Practice of Public Key Cryptography, Part II*, ser. Lecture Notes in Computer Science, J. Garay, Ed., vol. 12711. Virtual Event: Springer, Cham, Switzerland, May 10–13, 2021, pp. 528–558.

[13] J. Bossuat, R. Cammarota, J. H. Cheon, I. Chillotti, B. R. Curtis, W. Dai, H. Gong, E. Hales, D. Kim, B. Kumara, C. Lee, X. Lu, C. Maple, A. Pedrouzo-Ulloa, R. Player, L. A. R. Lopez, Y. Song, D. Yhee, and B. Yildiz, "Security guidelines for implementing homomorphic encryption," *IACR Cryptol. ePrint Arch.*, p. 463, 2024.

[14] Z. Brakerski, "Fully homomorphic encryption without modulus switching from classical gapsvp," in *CRYPTO*, ser. Lecture Notes in Computer Science, vol. 7417. Springer, 2012, pp. 868–886.

[15] L. D. Castro, "Verisimplepir," accessed: February 4, 2025. [Online]. Available: https://github.com/leodec/VeriSimplePIR

[16] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, "Private information retrieval," in *Proceedings of IEEE 36th Annual Foundations of Computer Science*, 1995, pp. 41–50.

[17] S. Colombo, K. Nikitin, H. Corrigan-Gibbs, D. J. Wu, and B. Ford, "Authenticated private information retrieval," in *USENIX Security Symposium*. USENIX Association, 2023, pp. 3835–3851.

[18] G. Cormode, M. Dall'Agnol, T. Gur, and C. Hickey, "Streaming zero-knowledge proofs," in *CCC*, ser. LIPIcs, vol. 300. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, pp. 2:1–2:66.

[19] G. Cormode, M. Mitzenmacher, and J. Thaler, "Practical verified computation with streaming interactive proofs," in *ITCS 2012: 3rd Innovations in Theoretical Computer Science*, S. Goldwasser, Ed. Cambridge, MA, USA: Association for Computing Machinery, Jan. 8–10, 2012, pp. 90–112.

[20] G. Cormode, J. Thaler, and K. Yi, "Verifying computations with streaming interactive proofs," *Proc. VLDB Endow.*, vol. 5, no. 1, pp. 25–36, 2011.

[21] H. Corrigan-Gibbs and D. Kogan, "Private information retrieval with sublinear online time," in *EUROCRYPT (1)*, ser. Lecture Notes in Computer Science, vol. 12105. Springer, 2020, pp. 44–75.

[22] N. Cui, X. Yang, B. Wang, J. Li, and G. Wang, "Svknn: Efficient secure and verifiable k-nearest neighbor query on the cloud platform*," in *ICDE*. IEEE, 2020, pp. 253–264.

[23] A. Davidson, G. Pestana, and S. Celi, "Frodopir: Simple, scalable, single-server private information retrieval," *Proc. Priv. Enhancing Technol.*, vol. 2023, no. 1, pp. 365–383, 2023.

[24] L. de Castro and K. Lee, "Verisimplepir: Verifiability in simplepir at no online cost for honest servers," in *USENIX Security Symposium*. USENIX Association, 2024.

[25] L. de Castro, K. Lewi, and E. Suh, "Whispir: Stateless private information retrieval with low communication," *IACR Cryptol. ePrint Arch.*, p. 266, 2024.

[26] M. Dietz and S. Tessaro, "Fully malicious authenticated PIR," in *CRYPTO (9)*, ser. Lecture Notes in Computer Science, vol. 14928. Springer, 2024, pp. 113–147.

[27] A. S. H. Encryption, "https://www.swift.org/blog/announcing-swift-homomorphic-encryption/ (Accessed Oct 2024)."

[28] L. Estimator, "https://github.com/malb/lattice-estimator."

[29] B. H. Falk, P. Mishra, and M. Shtepel, "Malicious security for PIR (almost) for free," *IACR Cryptol. ePrint Arch.*, p. 964, 2024.

[30] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *IACR Cryptol. ePrint Arch.*, p. 144, 2012.

[31] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology – CRYPTO'86*, ser. Lecture Notes in Computer Science, A. M. Odlyzko, Ed., vol. 263. Santa Barbara, CA, USA: Springer, Berlin, Heidelberg, Germany, Aug. 1987, pp. 186–194.

[32] D. Fiore, R. Gennaro, and V. Pastro, "Efficiently verifiable computation on encrypted data," in *ACM CCS 2014: 21st Conference on Computer and Communications Security*, G.-J. Ahn, M. Yung, and N. Li, Eds. Scottsdale, AZ, USA: ACM Press, Nov. 3–7, 2014, pp. 844–855.

[33] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *41st Annual ACM Symposium on Theory of Computing*, M. Mitzenmacher, Ed. Bethesda, MD, USA: ACM Press, May 31 – Jun. 2, 2009, pp. 169–178.

[34] C. Gentry, S. Halevi, and V. Vaikuntanathan, "i-Hop homomorphic encryption and rerandomizable Yao circuits," in *Advances in Cryptology – CRYPTO 2010*, ser. Lecture Notes in Computer Science, T. Rabin, Ed., vol. 6223. Santa Barbara, CA, USA: Springer, Berlin, Heidelberg, Germany, Aug. 15–19, 2010, pp. 155–172.

[35] S. Goldwasser, Y. T. Kalai, C. Peikert, and V. Vaikuntanathan, "Robustness of the learning with errors assumption," in *ICS*. Tsinghua University Press, 2010, pp. 230–240.

[36] Google, "Highway: Performance-portable, length-agnostic simd with runtime dispatch," accessed: February 4, 2025. [Online]. Available: https://github.com/google/highway

[37] ——, "Hintlesspir," accessed: February 4, 2025. [Online]. Available: https://github.com/google/hintless_pir

[38] K. Grisse, *Recommender Systems, Manipulation and Private Autonomy: How European Civil Law Regulates and Should Regulate Recommender Systems for the Benefit of Private Autonomy*. Springer International Publishing, 2023.

[39] S. Halevi and V. Shoup, "Faster homomorphic linear transformations in helib," in *CRYPTO (1)*, ser. Lecture Notes in Computer Science, vol. 10991. Springer, 2018, pp. 93–120.

[40] A. Henzinger, E. Dauterman, H. Corrigan-Gibbs, and N. Zeldovich, "Private web search with tiptoe," in *SOSP*. ACM, 2023, pp. 396–416.

[41] A. Henzinger, M. M. Hong, H. Corrigan-Gibbs, S. Meiklejohn, and V. Vaikuntanathan, "One server for the price of two: Simple and fast single-server private information retrieval," in *USENIX Security Symposium*. USENIX Association, 2023, pp. 3889–3905.

[42] A. Hoover, S. Patel, G. Persiano, and K. Yeo, "Plinko: Single-server PIR with efficient updates via invertible prfs," *IACR Cryptol. ePrint Arch.*, p. 318, 2024.

[43] U. how Live Caller ID Lookup preserves privacy, "https://developer.apple.com/documentation/sms_and_call_reporting/understanding_how_live_caller_id_lookup_preserves_privacy (Accessed Oct 2024)."

[44] M. M. Huang, B. Li, X. Mao, and J. Zhang, "Fully homomorphic encryption with efficient public verification," *IACR Cryptol. ePrint Arch.*, p. 1764, 2024.

[45] M. Kiraz and B. Schoenmakers, "A protocol issue for the malicious case of yao's garbled circuit construction," in *27th symposium on information theory in the Benelux*, vol. 29. IEEE, 2006, pp. 283–290.

[46] C. Knabenhans, A. Viand, A. Merino-Gallardo, and A. Hithnawi, "vfhe: Verifiable fully homomorphic encryption," in *WAHC@CCS*. ACM, 2024, pp. 11–22.

[47] E. Kushilevitz and R. Ostrovsky, "Replication is NOT needed: SINGLE database, computationally-private information retrieval," in *38th Annual Symposium on Foundations of Computer Science*. Miami Beach, Florida: IEEE Computer Society Press, Oct. 19–22, 1997, pp. 364–373.

[48] ——, "Replication is NOT needed: SINGLE database, computationally-private information retrieval," in *FOCS*. IEEE Computer Society, 1997, pp. 364–373.

[49] C. M. Learning and H. E. in the Apple Ecosystem, "https://machinelearning.apple.com/research/homomorphic-encryption (Accessed Oct 2024)."

[50] R. Lehmkuhl, A. Henzinger, and H. Corrigan-Gibbs, "Distributional private information retrieval," *IACR Cryptol. ePrint Arch.*, p. 132, 2025.

[51] B. Li, D. Micciancio, M. Raykova, and M. Schultz, "Hintless single-server private information retrieval," in *CRYPTO (9)*, ser. Lecture Notes in Computer Science, vol. 14928. Springer, 2024, pp. 183–217.

[52] J. Luo, X. Yi, F. Han, and X. Yang, "An efficient privacy-preserving recommender system in wireless networks," *Wirel. Networks*, vol. 30, no. 6, pp. 4949–4960, 2024.

[53] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Advances in Cryptology – EUROCRYPT 2010*, ser. Lecture Notes in Computer Science, H. Gilbert, Ed., vol. 6110. French Riviera: Springer, Berlin, Heidelberg, Germany, May 30 – Jun. 3, 2010, pp. 1–23.

[54] Y. Ma, K. Zhong, T. Rabin, and S. Angel, "Incremental offline/online PIR," in *USENIX Security Symposium*. USENIX Association, 2022, pp. 1741–1758.

[55] S. J. Menon and D. J. Wu, "SPIRAL: fast, high-rate single-server PIR via FHE composition," in *SP*. IEEE, 2022, pp. 930–947.

[56] ——, "YPIR: high-throughput single-server PIR with silent preprocessing," in *USENIX Security Symposium*. USENIX Association, 2024.

[57] M. H. Mughees, H. Chen, and L. Ren, "Onionpir: Response efficient single-server PIR," in *CCS*. ACM, 2021, pp. 2292–2306.

[58] M. Naor and B. Pinkas, "Oblivious transfer and polynomial evaluation," in *31st Annual ACM Symposium on Theory of Computing*. Atlanta, GA, USA: ACM Press, May 1–4, 1999, pp. 245–254.

[59] A. privacy-enhancing protocol and browser extension, "https://privacypass.github.io/ (Accessed Sep 2025)."

[60] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," in *STOC*. ACM, 2005, pp. 84–93.

[61] ——, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, pp. 34:1–34:40, 2009.

[62] R. L. Rivest, L. Adleman, M. L. Dertouzos *et al.*, "On data banks and privacy homomorphisms," *Foundations of secure computation*, vol. 4, no. 11, pp. 169–180, 1978.

[63] S. Servan-Schreiber, S. Langowski, and S. Devadas, "Private approximate nearest neighbor search with sublinear communication," in *SP*. IEEE, 2022, pp. 911–929.

[64] L. T. Thibault and M. Walter, "Towards verifiable FHE in practice: Proving correct execution of tfhe's bootstrapping using plonky2," *IACR Cryptol. ePrint Arch.*, p. 451, 2024.

[65] X. Wang and L. Zhao, "Verifiable single-server private information retrieval," in *ICICS*, ser. Lecture Notes in Computer Science, vol. 11149. Springer, 2018, pp. 478–493.

[66] Y. Wang, J. Zhang, J. Liu, and X. Yang, "Crust: Verifiable and efficient private information retrieval with sublinear online time," *IACR Cryptol. ePrint Arch.*, p. 1607, 2023.

[67] E. M. Williams and K. M. Carley, "Search engine manipulation to spread pro-kremlin propaganda," *Harvard Kennedy School (HKS) Misinformation Review*, 2023.

[68] L. Zhao, X. Wang, and X. Huang, "Verifiable single-server private information retrieval from LWE with binary errors," *Inf. Sci.*, vol. 546, pp. 897–923, 2021.

[69] M. Zhou, A. Park, W. Zheng, and E. Shi, "Piano: Extremely simple, single-server PIR with sublinear server computation," in *SP*. IEEE, 2024, pp. 4296–4314.

Appendix A. Additional Material

A.1. Notes on Persistent State

The client's persistent state in our vLHE scheme is $c \cdot \log q \cdot \sqrt{N}$ bits for a database with N entries and a purported infinity-norm bound p. Here, q is the ciphertext modulus of the underlying LHE, $\log q \approx 2 \log p + \log \sqrt{N}$, and c is defined as $\lceil \frac{\kappa}{\log q} \rceil$, where κ is the security parameter. We compare our persistent state with VeriSimplePIR [24] in Table 1.

VeriSimplePIR	Our Scheme
$> \boxed{n} \cdot \sqrt{N} \cdot \log(p^2\sqrt{N})$	$\approx \boxed{c} \cdot \sqrt{N} \cdot \log(p^2\sqrt{N})$

TABLE 1: Comparison of persistent state size. Here, n is the LWE lattice dimension parameter. Typically, we have $n \geq 1024$ and $c \leq 2$.

A.2. vLHE Security Proof and Additional Notes

Lemma A.1. *(Guessing an element in the nullspace; Lemma 2.4 [24]). Let $\ell, \kappa \in \mathbb{N}$ and $p \geq 2$. For all non-zero vectors $\boldsymbol{x} \in \mathbb{Z}_p^\ell$, the following holds:*

$$\Pr_{\boldsymbol{C} \xleftarrow{\$} \{0,1\}^{\kappa \times \ell}} [\ \boldsymbol{C} \cdot \boldsymbol{x} = \boldsymbol{0}\ ] \leq 2^{-\kappa}.$$

Lemma A.2. *(Correctness of Regev's LHE; [24]). Consider Regev's scheme $(\mathtt{Setup}, \mathtt{KeyGen}, \mathtt{Enc}, \mathtt{Dec})$. Let $p, \lambda \in \mathbb{N}$, $(n, q, \chi) \leftarrow \mathtt{Setup}(1^\lambda, p)$, where χ be the discrete Gaussian distribution over $\mathbb{Z}$ with standard deviation σ. Let $m, \ell \in \mathbb{N}$, $\delta \in (0,1)$, and $\boldsymbol{D} \in \mathbb{Z}^{\ell \times m}$ be a linear transformation. If $q \geq p \cdot \sigma \cdot \|\boldsymbol{D}\|_\infty \sqrt{2m \ln(2/\delta)}$, the following holds $\forall \mu \in \mathbb{Z}_p^m$:*

$$\Pr_{\mathtt{sk} \leftarrow \mathtt{KeyGen}(\mathtt{pp}), \mathtt{ct} \leftarrow \mathtt{Enc}_{\mathtt{sk}}(\mu)} [\mathtt{Dec}_{\mathtt{sk}}(\boldsymbol{D} \cdot \mathtt{ct}) = \boldsymbol{D} \cdot \mu \bmod p] \geq 1 - \delta$$

Theorem 3.11. *(Our protocol is vLHE). Figure 5 is a vLHE scheme (Definition 3.4) as long as $(n, m, q, (2\ell+1) \cdot B)$-SIS is hard*[15] *and LHE correctness*[16] *holds with overwhelming probability for linear transformations of ∞-norm $\leq 2B$.*

Proof. We begin by describing the construction of our simulator $\mathcal{S}$.

Simulator $\mathcal{S}$. The (stateful) simulator is constructed as follows:

- On message $(\mathsf{id}, \mathsf{Register})$ from $\mathcal{F}$:
  - Run the proof preprocessing phase from Figure 5 with the $\mathcal{A}$ to generate fresh $\boldsymbol{C}, \boldsymbol{Z}$. If it fails, output Abort to $\mathcal{F}$.
  - Store $(\mathsf{id}, \boldsymbol{C}, \boldsymbol{Z})$.
  - Run the extractor $\mathcal{E}$ from Lemma 2.4 to extract $\tilde{\boldsymbol{D}}$ by rewinding $\mathcal{A}$.
  - Send $\tilde{\boldsymbol{D}}$ to $\mathcal{F}$.
- On message $(\mathsf{id}, \mathsf{Query}, \perp)$ from $\mathcal{F}$:
  - If the output of $(\mathsf{id}, \mathsf{Register})$ was Abort, return Abort.
  - Run the online phase from Figure 5 with the $\mathcal{A}$ and set message $\boldsymbol{m} \leftarrow \boldsymbol{0}$. Find $(\mathsf{id}, \boldsymbol{C}, \boldsymbol{Z})$ in memory and use the corresponding $\boldsymbol{C}$ and $\boldsymbol{Z}$ for verification.
  - If any check fails, output Abort to $\mathcal{F}$. Otherwise, send Allow to $\mathcal{F}$.

Our goal is to prove that the distributions $\{\text{IDEAL}_{\mathcal{F},\mathcal{S}(z)}(X, \lambda)\}$ and $\{\text{REAL}_{\pi,\mathcal{A}(z)}(X, \lambda)\}$ are indistinguishable. These distributions have two parts: (1) output of the protocol (or output of $\mathcal{F}$ in the ideal world), and (2) output of $\mathcal{A}$. We construct a series of hybrids.

Hybrid H_0: This hybrid refers to the ideal world execution.

Hybrid H_1: In this hybrid, $\mathcal{S}$ replaces FHE encryption of $\boldsymbol{s}$ with FHE encryption of a different random key $\boldsymbol{s}'$ (independently sampled to $\boldsymbol{s}$), and then replaces the online phase check with $\boldsymbol{Z} \cdot [\boldsymbol{u}\ \boldsymbol{A}\boldsymbol{s}'] = \boldsymbol{C} \cdot [\boldsymbol{v}\ \boldsymbol{w}]$.

Proving H_0 and H_1 are indistinguishable: Appealing to the semantic security of the FHE scheme, the FHE encryption in the view of $\mathcal{A}$ is indistinguishable across the two hybrids. However, the verification outcome of $\boldsymbol{Z} \cdot \boldsymbol{A}\boldsymbol{s}' = \boldsymbol{C} \cdot \boldsymbol{w}$ can leak up to one bit about $\boldsymbol{s}'$. Since the encryption of LHE query is still done with the key $\boldsymbol{s}$, in H_1, the one-bit leakage is independent of the LHE ciphertext, while in H_0, the leakage is on the secret-key used to encrypt LHE query. Therefore, we additionally appeal the robustness of LWE under leaky secrets [35] for indistinguishability between encryption of LHE query with (leaky) $\boldsymbol{s}$ and (fresh) $\boldsymbol{s}'$.

Now we look at the protocol's output. If $\boldsymbol{w} \neq \boldsymbol{H} \cdot \boldsymbol{s}'$, i.e. the server didn't perform $\mathtt{FHE.Eval}_{\mathtt{evk}}(\boldsymbol{H}, \mathtt{ct}(\boldsymbol{s}'))$ correctly, $\mathcal{S}$'s online check will fail (Lemma 3.10) and it will send Abort to $\mathcal{F}$. This is identical behavior to H_0. The decryption result will be incorrect, but that doesn't affect the distributions because $\mathcal{F}$ doesn't use the decryption result of $\mathcal{S}$.

Remark A.3 (Decryption oracle)**.** In situations where feedback on acceptance of the server's response is provided to the server, it can be used as a decryption oracle that can leak at most one-bit of information about $\boldsymbol{s}$ (LHE secret key used in the online phase) via the decryption operation $\boldsymbol{w} \leftarrow \mathtt{FHE.Dec}_{\mathtt{sk}}(\mathtt{ct}(\boldsymbol{H}\boldsymbol{s}))$. Since $\boldsymbol{s}$ is an ephemeral key (refreshed each request), this doesn't leak information about the client's query because at most one-bit of entropy is lost and LWE is robust against such leaky secrets [35]. When picking FHE parameters, this loss can be accounted for to ensure at least λ bits of security.

Hybrid H_2: Next, we allow $\mathcal{F}$ to reveal the full query message $(\mathsf{id}, \mathsf{Query}, \boldsymbol{x})$ to $\mathcal{S}$, and $\mathcal{S}$ encrypts $\boldsymbol{x}$ rather than encrypting 0.

Proving H_1 and H_2 are indistinguishable: $\mathcal{A}$'s view is indistinguishable from the semantic security of Regev's LHE scheme. Output of $\mathcal{F}$ is unaffected if $\mathcal{S}$'s plaintext query is changed (Lemma 3.10; any cheating by the server and simulator's checks fail), or its decryption result is changed.

Hybrid H_3: We now revert back to using FHE encryption of $\boldsymbol{s}$ (the key used to encrypt LHE query), and revert back the original check in the online phase.

Proving H_2 and H_3 are indistinguishable: Follows similarly to the previous hybrids.

Hybrid H_4: In this hybrid, the $\mathcal{S}$ runs the full online phase as in Figure 5 and sends its full output (i.e. decryption result and Abort or Allow) to $\mathcal{F}$. We change $\mathcal{F}$ to output what $\mathcal{S}$ says for each query message $(\mathsf{id}, \mathsf{Query}, \boldsymbol{x})$.

Proving H_3 and H_4 are indistinguishable: The view of $\mathcal{A}$ is unaffected because the hybrids only differ in what $\mathcal{F}$

15. When $\boldsymbol{H}$ is honestly generated using a database $\boldsymbol{D}$, this can be reduced to $(n, m, q, (B + \ell \cdot \|\boldsymbol{D}\|_\infty))$-SIS.

16. Please refer to Lemma A.2 for details on the correctness condition for underlying LHE.

outputs. Moreover, the Abort pattern is the same. The only change is that when $\mathcal{S}$ would send Allow in H_3, it now additionally sends the decrypted output. Since Allow is only sent when all the checks pass, using Lemma 3.10, we have that $\boldsymbol{v} = \tilde{\boldsymbol{D}} \cdot \boldsymbol{u}$ and $\boldsymbol{w} = \boldsymbol{H} \cdot \boldsymbol{s}$, where $\boldsymbol{H} = \tilde{\boldsymbol{D}} \cdot \boldsymbol{A}$, and $\tilde{\boldsymbol{D}}$ is the same as $\boldsymbol{D}_{\mathsf{id}}$ extracted by $\mathcal{S}$ and then stored by $\mathcal{F}$ when the previous request $(\mathsf{id}, \mathsf{Register})$ was issued. Therefore, from the correctness of LHE (Lemma A.2), we have that the output of $\mathcal{F}$ is same across the two hybrids. Therefore, they are indistinguishable.

Hybrid H_5: This refers to the real world execution.

Proving H_4 and H_5 are indistinguishable: the hybrids are identical. □

Remark A.4 (Anonymity in vLHE)**.** In the proof of Theorem 3.11, the simulator $\mathcal{S}$ is given the id for each online query. To be compatible with anonymity, the ideal functionality $\mathcal{F}$ no longer provides id to $\mathcal{S}$ for Query messages. In this case, $\mathcal{S}$ goes through all $(\mathsf{id}, \boldsymbol{C}, \boldsymbol{Z})$ triples that it has from Register messages and uses each one of them to verify. Let L denote the list of id for which the verification passes. Then $\mathcal{S}$ sends L to $\mathcal{F}$, and $\mathcal{F}$ checks if the actual id of the Query message lies in L. If it does, then $\mathcal{F}$ treats this as Allow as proceeds as before (Figure 3). Otherwise, it is an Abort and it returns $\perp$ to the client.

A.3. Strong cvLHE

Strong cvLHE (scvLHE). Our definition of cvLHE (§4) can be made stronger by requiring publicly-verifiable stateful cheating not just for the online phase, but the entire interaction between a stateful client and the server. In particular, the ConfirmCheater algorithm takes the *entire* protocol transcript τ, offline state $\mathtt{st}$ of the stateful client, and any auxiliary information α from the server.

scvLHE construction overview. We only require changes to the offline phase compared to our cvLHE protocol. Each message includes a CR hash of the transcript so far; the entire message is signed by sender and verified by the receiver (aborting on verification failure). Additionally, in the first message from the client to the server, the client sends its signature public key and commits to its offline-phase FHE secret key and encryption error. The client stores this one-time transcript in its infrequently-accessed storage (e.g. cloud storage or external disk), excluding $\boldsymbol{H}$ from it.

Transcript storage in scvLHE. To prove that the server cheated, a stateful client needs to present the appropriately signed transcript. For each stateful client, this includes their one-time preprocessing transcript and it has to be stored so that it can be furnished if the server cheats in a future online phase. This storage is about 70 MiB for an 8 GiB database. Either the server can be tasked with storing this transcript while the client only stores an authentication tag signed by the server, or the client can offload it to a separate remote storage itself. Unlike the persistent state in VeriSimplePIR, this transcript is not needed when making online queries, and only rarely fetched—when the server is caught cheating.

scvLHE ConfirmCheater *(doesn't require* $\boldsymbol{D}$*).* It gets as input the complete transcript τ, where the server provides $\boldsymbol{H}$ and the client provides the rest of the transcript. Additionally, it gets as input the server's signature public key (from PKI), and as part of $\mathtt{st}$, an opening to the commitment of client's offline phase FHE secret key and the error used for (offline) encryption (protects honest server from getting framed). The algorithm proceeds as follows:

- It first verifies all the signatures in τ are valid, and verifies that the transcript is consistent w.r.t. CR hashes of previous messages. If verification fails, output 0[17].
- If $\boldsymbol{H}$, provided by the server, is inconsistent with its signed CR hash from the transcript, output 1.
- It uses client-provided openings of the commitments of the offline-phase FHE secret key and encryption error.
- If the encryption error is outside the permitted error distribution, output 0.
- It decrypts $\boldsymbol{C}, \boldsymbol{Z}$ from $\mathtt{ct}(\boldsymbol{C}^T), \mathtt{ct}(\boldsymbol{Z}^T)$ in τ using FHE secret key.
- If any of these checks fail, output 1; else, output 0:
  - $\boldsymbol{Z} \cdot [\boldsymbol{A}\ \boldsymbol{u}] = \boldsymbol{C} \cdot [\boldsymbol{H}\ \boldsymbol{v}]$ and $\|\boldsymbol{Z}\|_\infty \leq B$ (Figure 5)
  - $\mathtt{ct}(\boldsymbol{H}\boldsymbol{s})$ in server's response is the output of $\mathtt{FHE.Eval}_{\mathtt{evk}}(\boldsymbol{H}, \mathtt{ct}(\boldsymbol{s}))$; $\mathtt{FHE.Eval}$ is deterministic.

Since the online phase of our cvLHE protocol can't have client signatures for anonymity, nothing binds the online phase to a particular client's identity unlike the offline phase which is bound by the client's public key (signed outgoing messages). This opens the door for mix-and-match transcripts where one client uses another client's offline transcript with its own online transcript to prove that a server cheated. This is not an issue because the online and offline transcripts in our protocol are fully decoupled. An honest server uses the same $\boldsymbol{D}$ for all clients and the offline phase outputs $(\boldsymbol{C}, \boldsymbol{Z})$ are not tied to any specific client.

A.4. Analysis of cvLHE

We model interaction between server S and n clients $i \in [n]$.

Definition A.5 (Game Model)**.** *Each client* i *chooses an action* $a_i \in \{\mathsf{SF}, \mathsf{SL}\}$*, where* SF *means* stateful *and* SL *means* stateless*. Let* $m = |\{i : a_i = \mathsf{SF}\}|$ *denote the number of stateful clients. The server chooses* $s \in \{\mathsf{H}, \mathsf{C}\}$*, where* H *means* honest *and* C *means* cheat*. The server cannot observe a client's action, only the distribution of types.*

Parameters:

- *Server's gain from cheating against an* SL *request:* $B > 0$.
- *Server's reputation penalty if caught cheating against an* SF *request:* $F > 0$.
- *Resource cost to a stateful client per request:* $c > 0$.
- *Privacy loss to a stateless client if cheated:* $L > 0$.

Payoffs:

17. Failing server signatures mean that either the client is altering server-signed messages in τ or the server is denying service (outside our threat model) by sending wrong signatures; we output 0 in both cases.

- *If $s = \mathsf{H}$: server payoff 0; SF client payoff $-c$; SL client payoff 0.*
- *If $s = \mathsf{C}$: server payoff B for each SL request and $-F$ for each SF request; SF client payoff $-c$; SL client payoff $-L$.*

Let $p^* = \frac{B}{B+F}$ and $m^* = \lceil p^* \cdot n \rceil$.

Lemma A.6 (Server cutoff rule)**.** *Given m stateful clients (so that the probability of a request being stateful is $p = m/n$), the server's unique best response is*

$$s = \mathsf{H} \quad \textit{iff} \quad m \geq m^*, \qquad s = \mathsf{C} \quad \textit{iff} \quad m < m^*.$$

Proof. If the server cheats, expected payoff is $(1-m/n)B - (m/n)F$. This is non-positive exactly when $m/n \geq p^*$. □

Proposition A.7 (Asymmetric pure equilibria)**.** *Suppose $c < L$. For any subset $M \subset [n]$ of size m^*, when exactly clients in M choose SF and all others choose SL, together with server strategy $s = \mathsf{H}$, forms a pure Nash equilibrium.*

Proof. With m^* stateful clients, Lemma A.6 implies that honesty is the server's best response.

- Any SL client deviating to SF would incur cost $c > 0$ without changing the outcome (server remains honest).
- Any SF client deviating to SL would reduce m below m^*, causing the server to cheat. That client's payoff would change from $-c$ to $-L$, which is worse ($L > c$).

Thus no player can profitably deviate. □

We now argue why the critical mass of stateful clients relative to the entire client population (or an upper bound on client population) is typically quite small, and why $c < L$.

- m^* **is small because typically** $B \ll F$. In a cvPIR scheme, publicly-verifiable cheating makes the cost of getting caught extremely high, i.e. F is very large. Moreover, the gain B from a *single* cheating against a stateless client is dwarfed in comparison because a single cheating only allows the server to mount a *single* selective failure attack, which doesn't even leak the full query of a single client. The exact ratio between B, F will depend on the application; $\frac{1}{100}$ or even less seems very reasonable.
- **Privacy-sensitive clients justify** $c < L$. In our model, we treat the cost profiles (c, L) the same for each client. The security analysis of (c)vPIR is only relevant when the system has at least one client, and therefore, $c < L$. The resource cost c for our scheme is lower than VeriSimplePIR, and hence, offers better payoff and makes it easier to satisfy $c < L$.

A.5. Notes on Implementation and Evaluation

Beyond our scope. We leave the implementation of our streaming optimization, and experimentation without the honest digest assumption for future work. Implementation of vPNNS is out of the scope of this work. HintlessPIR suggests two optimizations to reduce the server's response size - (1) include the first component of server's RLWE-based response ciphertext in public parameters, and (2) compress the response further by switching to a smaller modulus since the response ciphertext no longer needs to support homomorphism. While both of these are compatible with our protocol, they are not supported in HintlessPIR's code, and therefore, not included in our code. We do not optimize parts of preprocessing phase (e.g. expansion time for LWE matrix). Recent optimizations [50], [56] that cast RLWE as LWE ciphertexts with mod switch can improve these parts. We leave their implementation to future work.

Database sizes. We evaluate on databases of varying sizes. For each database, we also report the 2D layout we used for our code and for HintlessPIR baseline; layout is a tuple (no. of rows, no. of cols). Our layout prioritizes lower online communication and client storage. When the size of each entry is more than a byte, we use the standard "stacking" optimization of splitting it over contiguous one-byte cells in the column [41], [51]. For VeriSimplePIR, layout selection is done internally by the code to optimize communication.

- 256 MiB: 2^{20} entries, each 2^8 B, with layout $(2^{14}, 2^{14})$.
- 512 MiB: 2^{26} entries, each 8 B, with layout $(2^{15}, 2^{14})$.
- 1 GiB: 2^{30} entries, each 1 B, with layout $(2^{15}, 2^{15})$.
- 8 GiB: 2^{18} entries, each 32 KiB, with layout $(2^{17}, 2^{16})$.
- 16 GiB: 2^{34} entries, each 1 B, with layout $(2^{17}, 2^{17})$.

Reducing preprocessing communication and improving throughput with low client storage. We also evaluate a wide matrix layout that optimizes preprocessing communication and server throughput over client storage and online communication. This improves the query-level preprocessing communication by $\approx 3\times$ and server throughput by up to $1.2\times$ while client's persistent state stays low: 2 MiB for 8 GiB DB and 4 MiB for 16 GiB DB.

- 8 GiB Wide: 8 GiB as above with layout $(2^{15}, 2^{18})$.
- 16 GiB Wide: 16 GiB as above with layout $(2^{15}, 2^{19})$.

Performance breakdown. Table 2 shows a performance breakdown of query-level preprocessing and online phase.

One-time preprocessing. There are two parts to one-time preprocessing - (1) database-level preprocessing (DLP) that is independent of the number of clients, and (2) client-level preprocessing (CLP) that is done once for each client. Of the two, the efficiency of CLP is more important because the former gets amortized over the number of clients (potentially several thousands). It takes 18k sec (resp. 1k sec) for the server to do DLP for our vPIR scheme for a 16 GiB (resp. 1 GiB) database compared to 3k sec (resp. 231 sec) for HintlessPIR[18]. Coming to CLP, the total computation done by each client with the server is around 673 sec (resp. 136 sec) for our scheme vs 664 sec (resp. 78 sec) for VeriSimplePIR[19] for a 16 GiB (resp. 1 GiB) database. We note that with an optimized implementation, the cost of one-time preprocessing can come down by significant amount;

18. We didn't evaluate the DLP time for VeriSimplePIR, but it's expected to be similar to our vPIR scheme.

19. It is zero for HintlessPIR.

DB Size	Computation Time (s|ms) Server, Client						Communication (KiB) ↑, ↓				
	Non-verifiable		Verifiable				Non-verifiable		Verifiable		
	HintlessPIR		VeriSimplePIR		This Work		HintlessPIR		VeriSim.	This Work	
	Prep[*]	*Online*	*Prep*[*]	*Online*	*Prep*[*]	*Online*	*Prep*[**]	*Online*	*Online*	*Prep*[**]	*Online*
256 MiB	0.5, 1.7	30, 0.8	0, 2.8	30, 61.1	0.1, 1.6	60, 2.2	360, 2.9k	79, 79	72, 73	952, 896	79, 79
512 MiB	0.6, 1.2	70, 0.8	0, 4.3	80, 94.2	0.3, 2.6	120, 2.9	360, 5.8k	79, 157	111, 111	952, 1.8k	79, 158
1 GiB	0.6, 3.8	140, 1.5	0, 5.7	140, 126	0.3, 5.7	240, 4.4	360, 5.8k	157, 158	148, 148	952, 1.8k	157, 158
8 GiB	1.7, 5.2	1.2k, 3.0	0, 15.8	1.1k, 853	0.8, 11.1	2k, 12.2	360, 23k	315, 632	410, 410	952, 7.2k	316, 631
Wide DB	0.6, 36.6	1.2k, 11.3			0.3, 51.3	2k, 26.1	360, 5.8k	1.3k, 157		952, 1.8k	1.3k, 158
16 GiB	1.7, 11.0	2.4k, 5.7	0, 22.7	2.3k, 497	0.8, 14.3	3.9k, 17	360, 23k	631, 631	591, 591	952, 7.2k	631, 633
Wide DB	0.6, 72.1	2.4k, 23			0.3, 93.1	3.9k, 49	360, 5.8k	2.5k, 158		952, 1.8k	2.5k, 157

Beyond the scope of our implementation (further optimizing the preprocessing phase above):
[*] Client's time for preprocessing (Prep phase) can be improved across all works (e.g. expansion of LWE matrix) with recent optimizations [50], [56].
[**] Download in preprocessing (Prep phase) in our work & HintlessPIR can be further reduced by about $4\times$ with optimizations from HintlessPIR [51].

TABLE 2: Performance breakdown after one-time preprocessing. Computation (resp. communication) costs show server (resp. client ↑) and client (resp. client ↓) cost, respectively. Computation cost is in secs for preprocessing and in ms for online. For all PIR schemes here, we use query-level preprocessing (labeled *Prep*). Prep comm. for VeriSimplePIR is zero. About 88-98% of client preprocessing time for prior work and 62-74% for our work goes in expansion of LWE matrix.

some optimizations for the client compute involve leveraging SIMD (currently used only for server-side computation), faster matrix multiplications when one operand is *ternary* (e.g. multiplication with secret vectors), and replacing the costly Gaussian error sampling in encryption with a centered Binomial distribution. Lastly, the total communication during CLP (per client) is comparable between our scheme and VeriSimplePIR. It is 80 MiB (resp. 20 MiB) for us vs 74 MiB (resp. 17 MiB) for VeriSimplePIR for 16 GiB (resp. 1 GiB) database.

Further evaluation. For completeness, we compare our verifiable PIR scheme with non-verifiable hint-free YPIR+SP, even though the two are incomparable in terms of security. We use wide 8 GiB and 16 GiB databases. Throughput of YPIR is only 1.2-1.4$\times$ better than our vPIR and total communication 2.8-3$\times$ better, while providing weaker security.

A.6. Verifiable Symmetric PIR (vSPIR)

Our vPIR scheme can be combined with a verifiable oblivious pseudorandom function (VOPRF) to construct a symmetric vPIR protocol. A symmetric PIR protocol additionally provides database privacy to the server against malicious clients who might try to access more than one database record in a single PIR request.

Following the standard approach to construct symmetric PIR in the non-verifiable setting [58], we present our protocol below. Denote the VOPRF output on input i and key k as: $f_k(i)$.

- Given the database $\boldsymbol{D}$ and key k, for all i, the server encrypts the i-th record in the database $\boldsymbol{D}$ with the key $f_k(i)$. Denote this encrypted database as $\boldsymbol{D'}$.
- The client issues vPIR queries on the encrypted database $\boldsymbol{D'}$ and recovers an encrypted record.
- Alongside the vPIR query, client makes a VOPRF query with input i to learn the record-specific decryption key $f_k(i)$.

A VOPRF binds the server to a unique key k, and therefore, the decryption of $\boldsymbol{D'}[i]$ yields $\boldsymbol{D}[i]$, as desired, and otherwise the client detects the cheating.

Appendix B. Meta-Review

The following meta-review was prepared by the program committee for the 2026 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

B.1. Summary

This work considers the problem of private information retrieval (PIR), and proposes a new verifiable PIR scheme based on standard lattice assumptions. The key innovation in the proposed scheme is a reduction in the storage cost for the clients and the removal of the random oracle assumption from prior works.

Beyond reducing client storage, the paper introduces the concept of covert vPIR (cvPIR), where stateful clients benefit from full vPIR security, and even stateless clients gain covert security against a malicious server

B.2. Scientific Contributions

- Reduces client storage costs in verifiable PIR with preprocessing
- Proposes a covert model to support stateless clients

B.3. Reasons for Acceptance

1) Reducing client storage costs is important in a world where clients will have access to many PIR databases, given that existing schemes had very large costs.
2) The notion of stateless clients and the type of guarantees they get is an interesting direction for this type of work.

B.4. Noteworthy Concerns

The significance of reducing storage cost pales in comparison with all of the other issues with preprocessing-based PIR (low throughput, struggles to handle dynamic databases, high communication during preprocessing).